



Deep Learning Optimized - Jean Zay

Introduction – Jean Zay – GPU



INSTITUT DU
DÉVELOPPEMENT ET DES
RESSOURCES EN
INFORMATIQUE
SCIENTIFIQUE



DLO-JZ presentation

IDRIS ◀

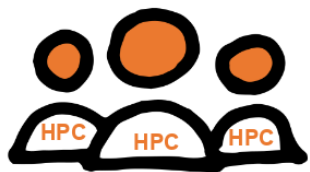
Agenda ◀

Presentation of the participants ◀

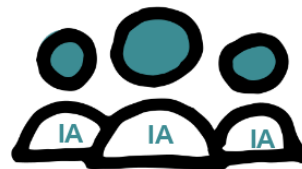
System



User Assistance



11 ingénieur·e·s



12 ingénieur·e·s

BLOOM on Jean Zay

Pre-training :
117 days (2022)
384 x 80GB A100 GPUs



Train modulaire des formations IDRIS

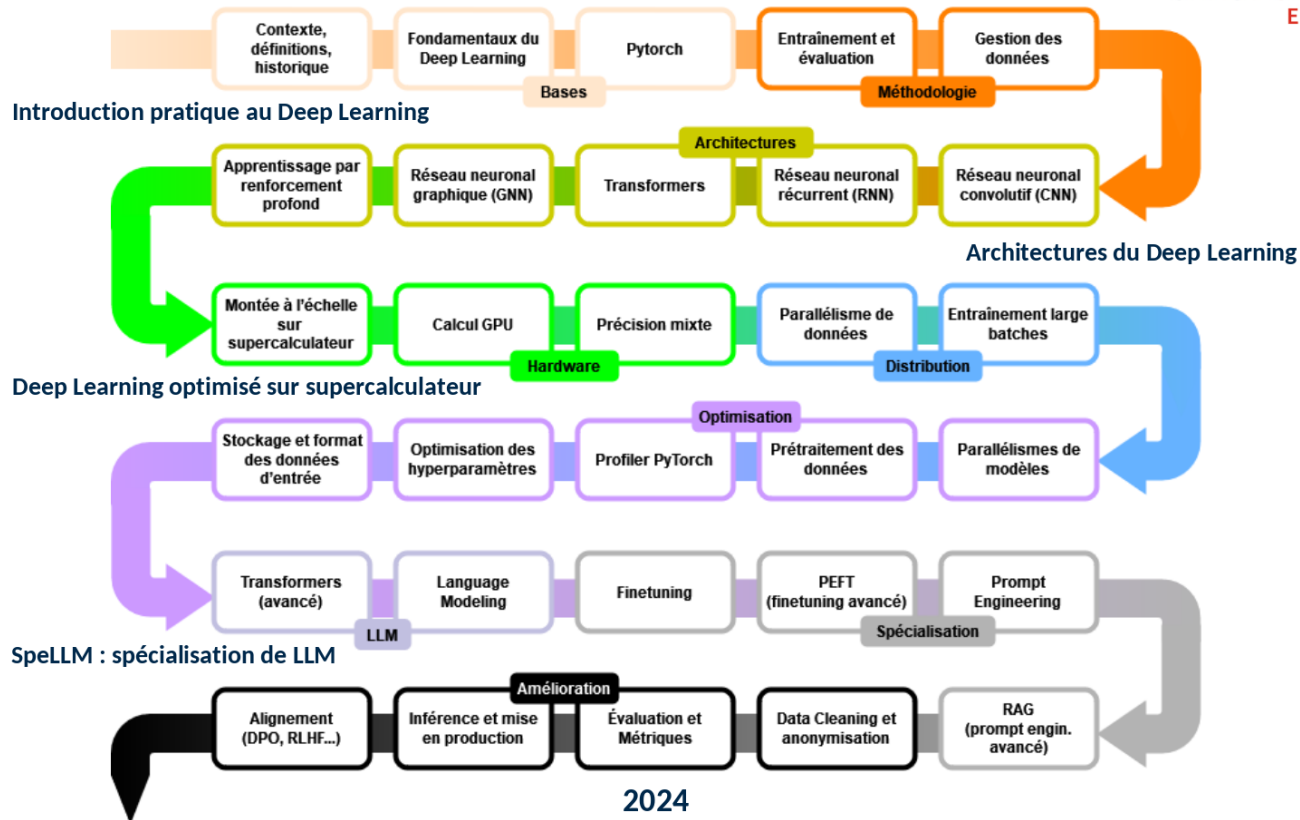


IDRIS

Les formations IA

Pour les inscriptions ou une formation sur mesure contacter

CNRS FORMATION
ENTREPRISES





DLO-JZ

Agenda – Covered topics

Day 1

- Presentation of Jean Zay
- The challenges of scaling up
- **GPU computing + Mixed Precision**
- Tensor optimisation
- The PyTorch profiler

Day 2

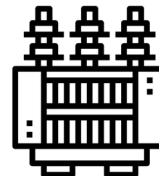
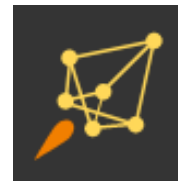
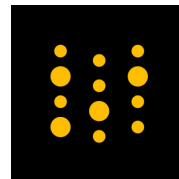
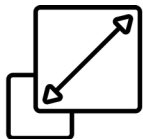
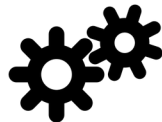
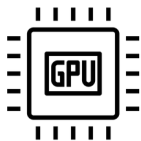
- **DataLoader optimizations**
- **Distribution + Data Parallelism** ❤️
- **Data Augmentation**

Day 3

- Data storage and formats
- **Training and large batches**
- JIT

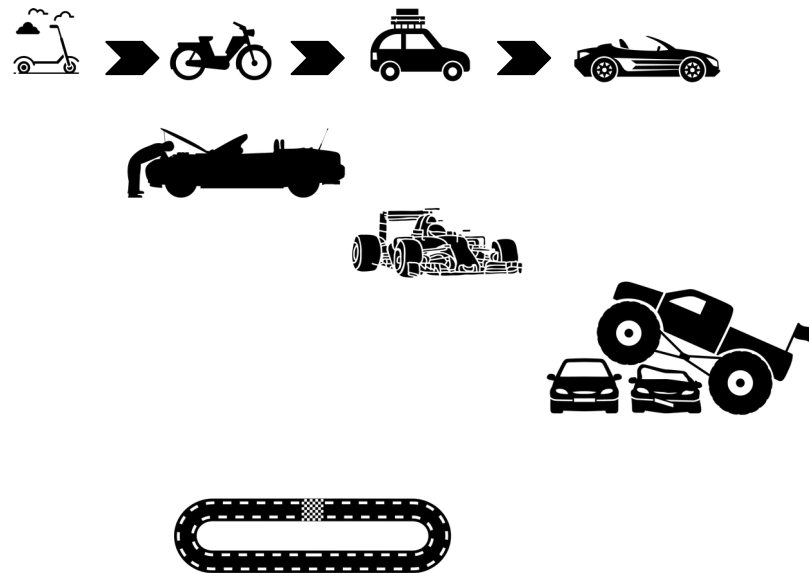
Day4

- Good practices
- Large Model Scaling + FSDP
- **Model parallelisms**
- Visualization tools
- HyperParameter Optimization



Practical Workshop

- Day 1, 2, 3, 4 :
 - System Optimization : GPU, Mixed Precision
 - Profiler
 - DataLoader
 - Distributed Data Parallelism
 - Data Augmentation
 - *Optimizers* & LR scheduler
 - torch.compile & torchscript
 - *FSDP*
 - *Hyper-Parameters Optimization (HPO)*



- Reservation partition A100 JZ

Presentation of participants



Jean Zay

Supercomputer ◀

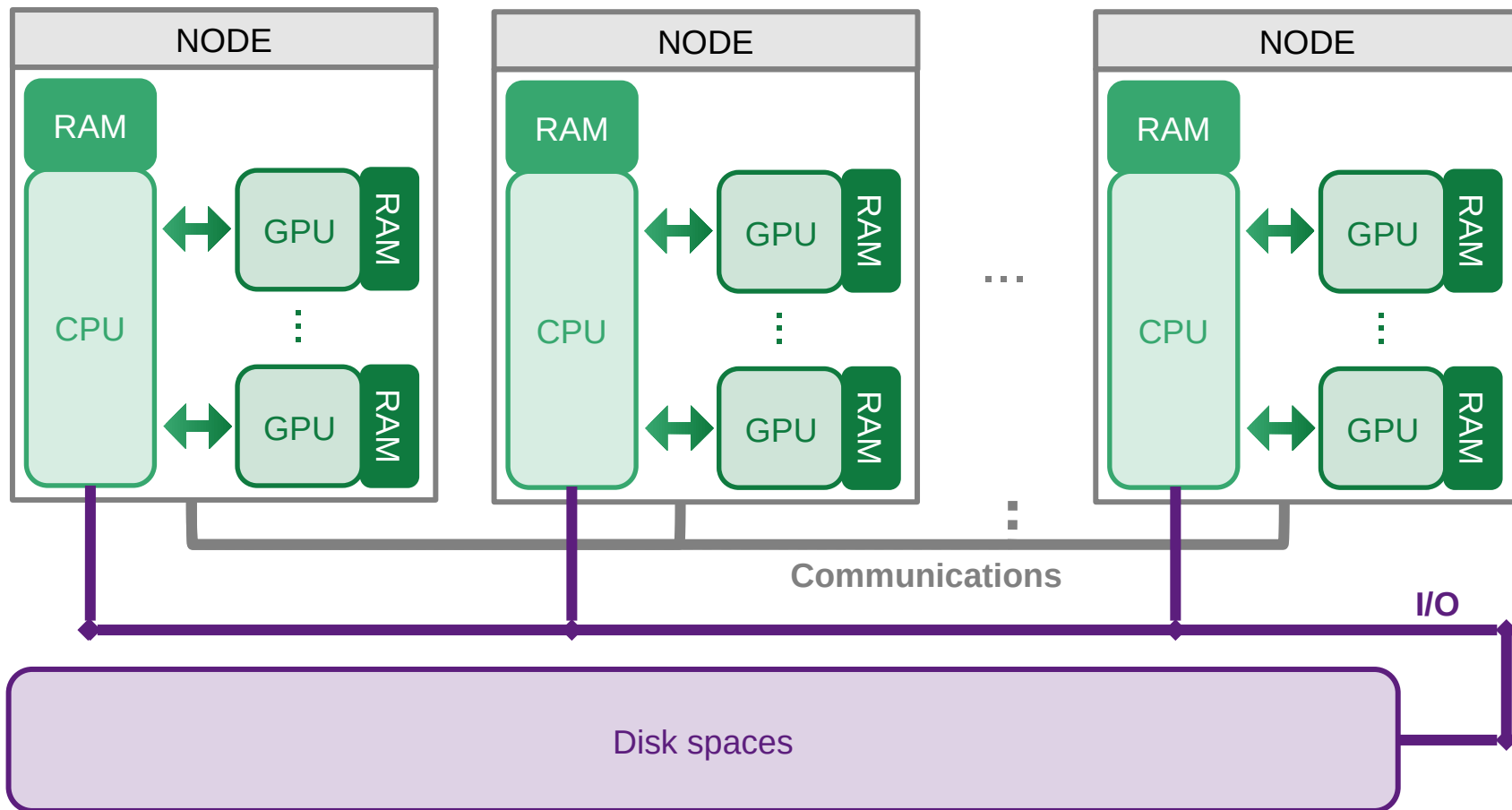
Jean Zay ◀

Job submission with Slurm ◀

JupyterHub on Jean Zay ◀

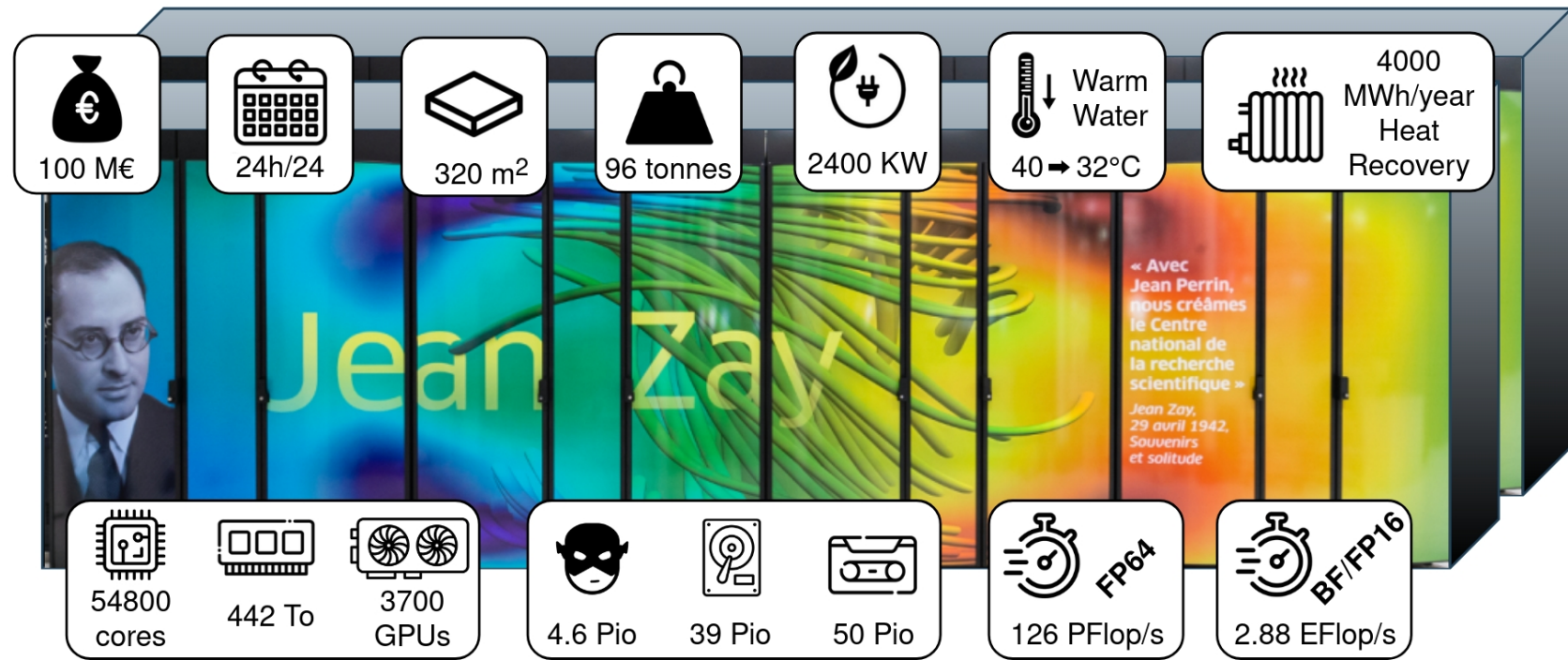
Slurm tools for python notebooks ◀

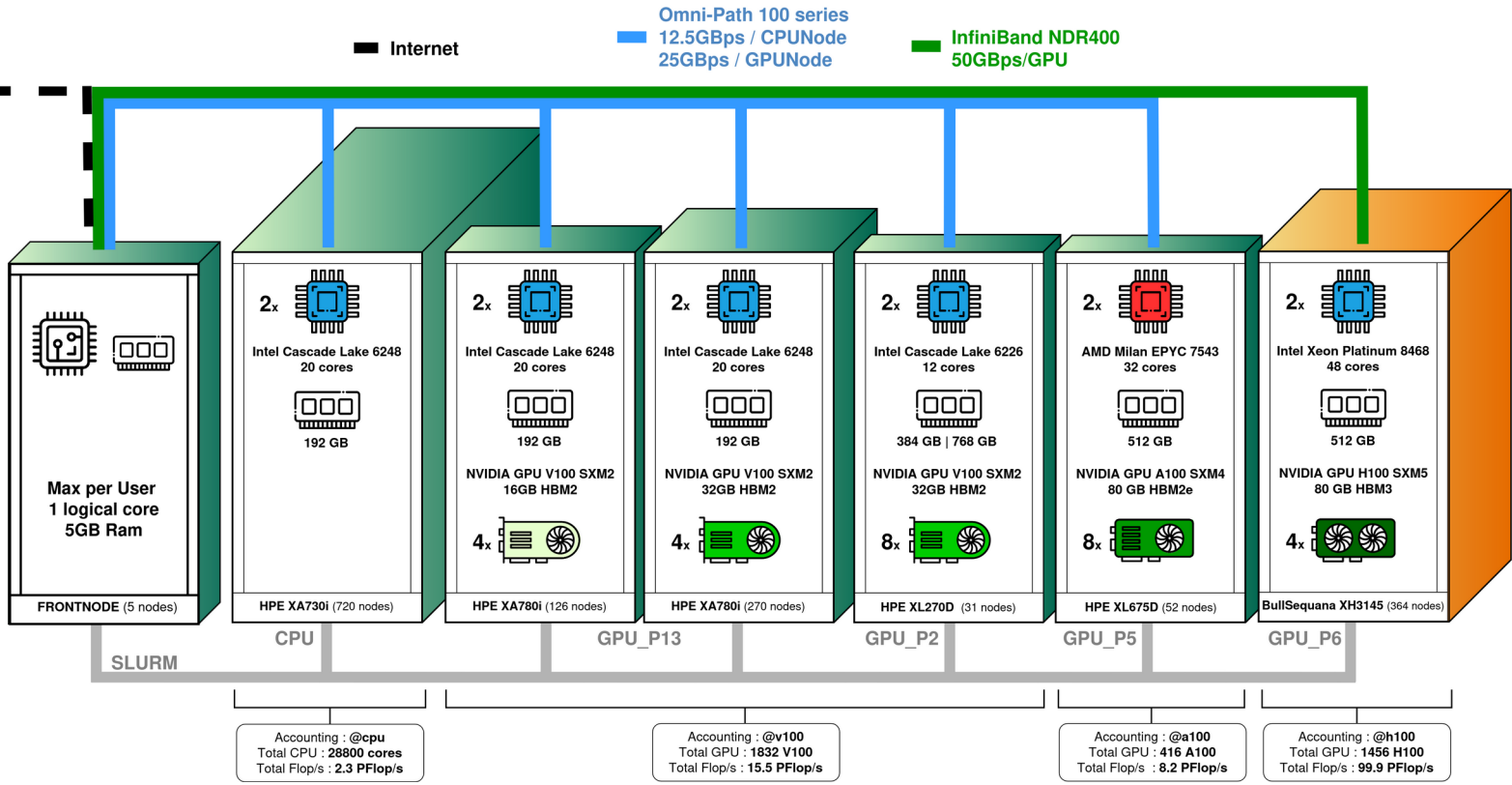
What's a supercomputer?

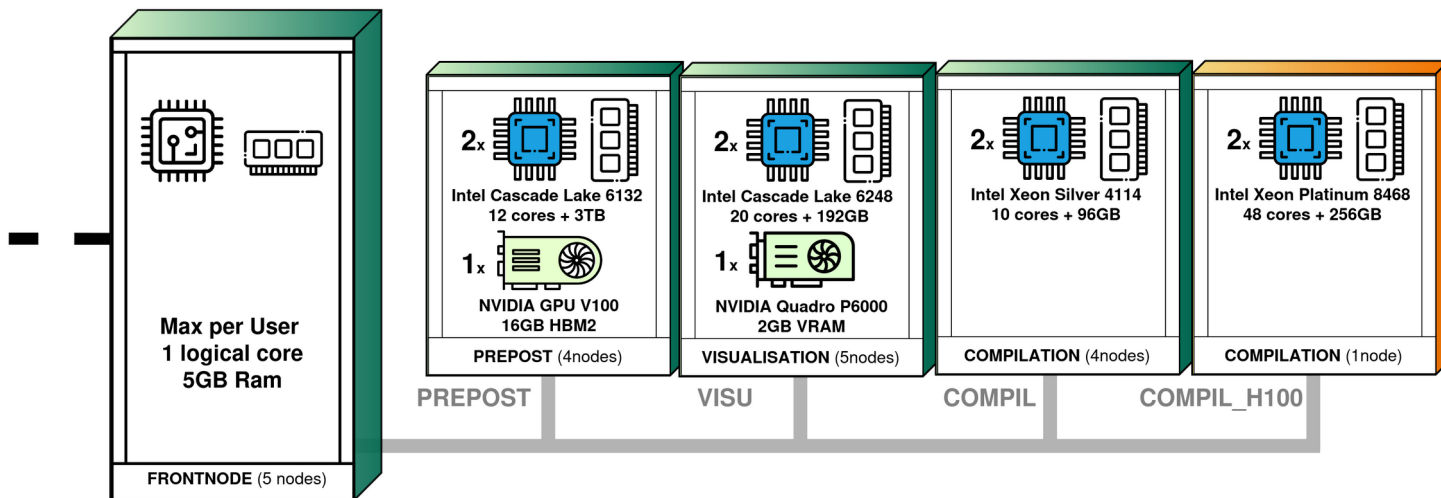


Jean Zay

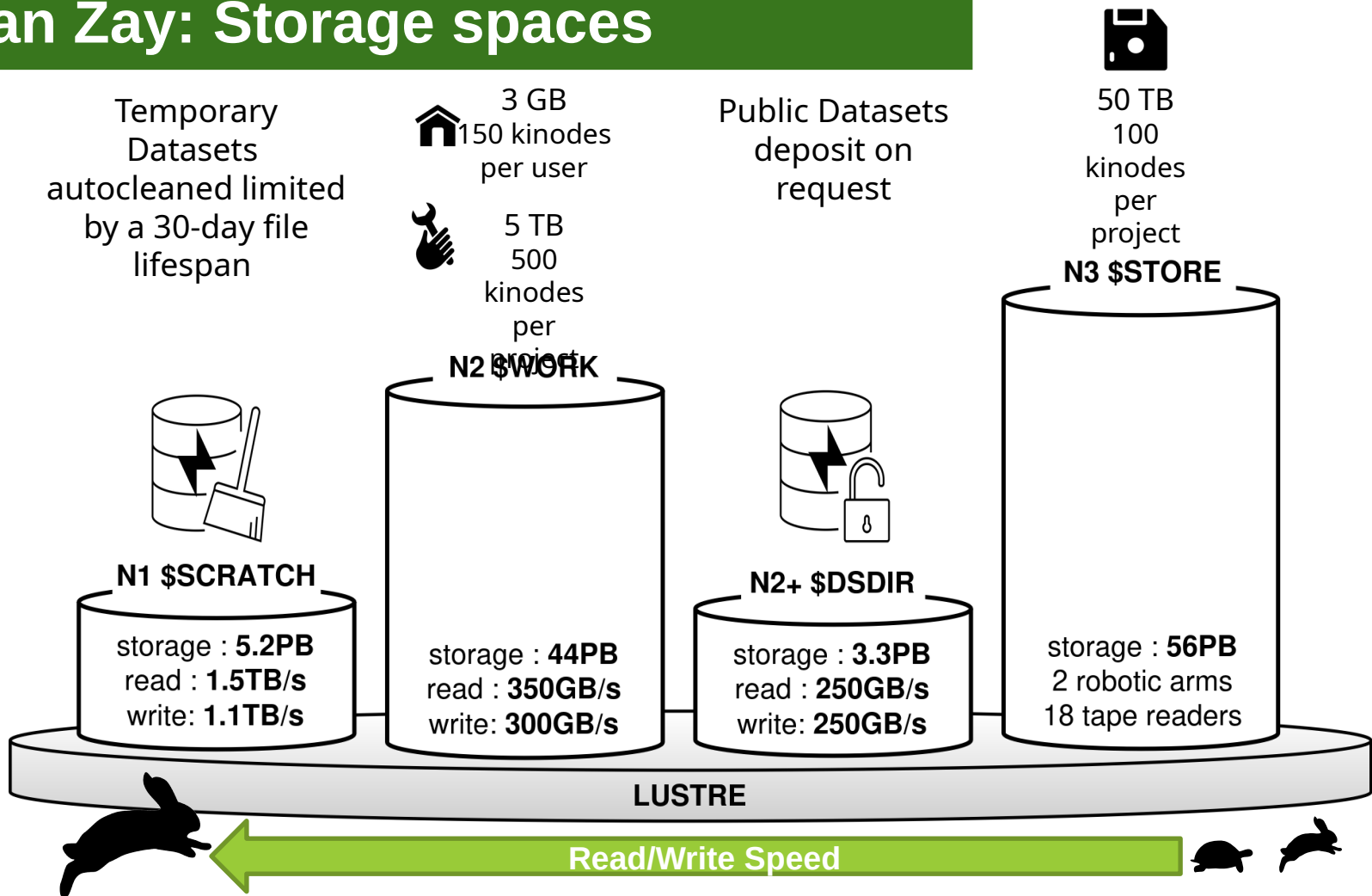
First national converged supercomputer dedicated to Artificial Intelligence (AI) and High Performance Computing (HPC)







Jean Zay: Storage spaces

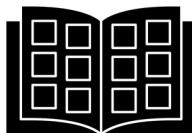




Catalogue of shared modules

(conda environments)

- Installed by IDRIS
- Completed on request



```
login@jean-zay3:~$ module load pytorch-gpu/py3/1.11.0
Loading requirement: ...
(pytorch-gpu-1.11.0+py3.9.12) login@jean-zay3:~$
```

- Customizable

```
~$ pip install --user --no-cache-dir <paquet>
```



Conflicts between versions
Storage spaces saturation

Personal conda environments

```
login@jean-zay3:~$ module load anaconda-py3/2023.03
(base) login@jean-zay3:~$ conda create -n myenv
```



Storage spaces saturation ++

Singularity containers

```
login@jean-zay3:~$ module load singularity
```

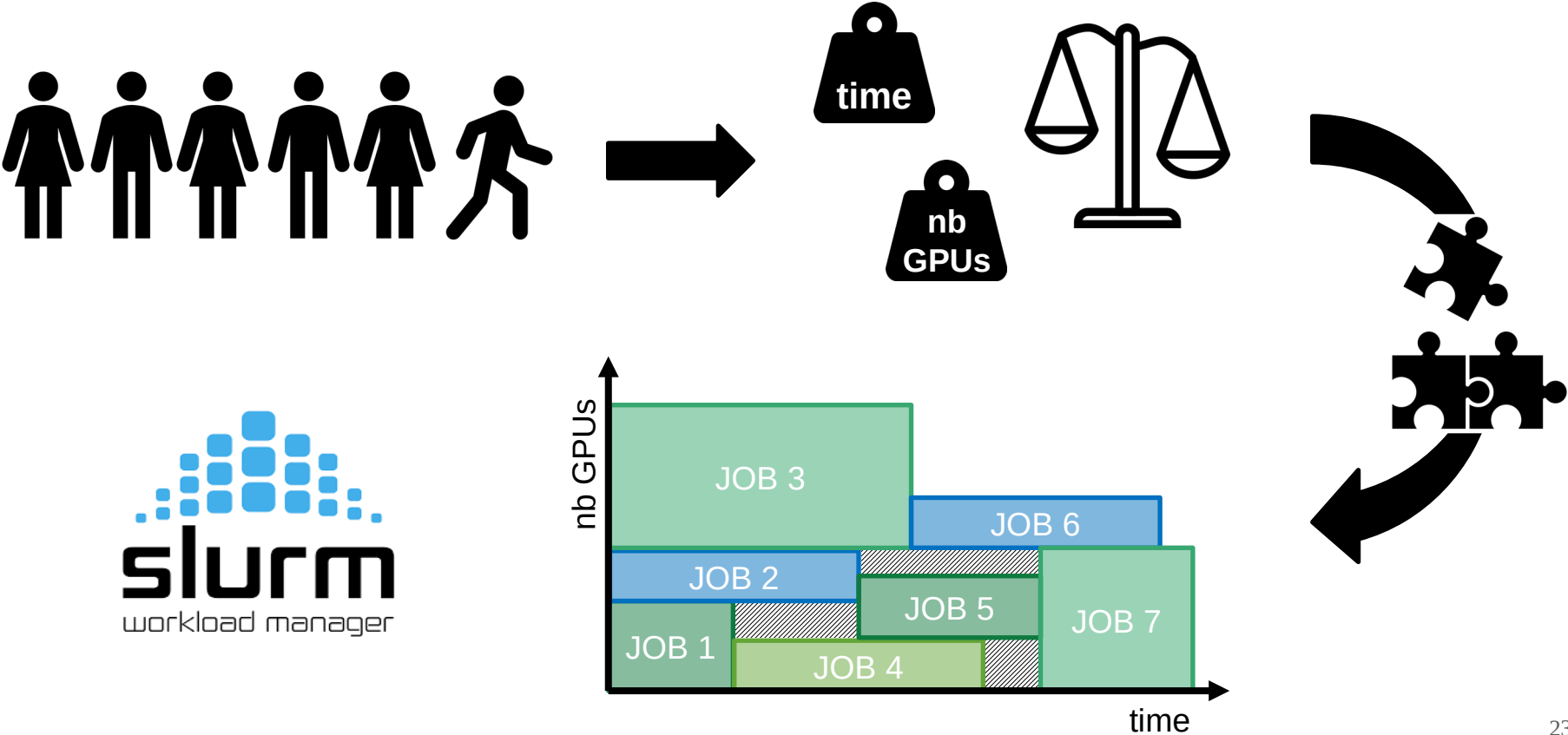


Import SIF images on Jean Zay

- From your PC
- From public deposits
- Possible conversion from docker

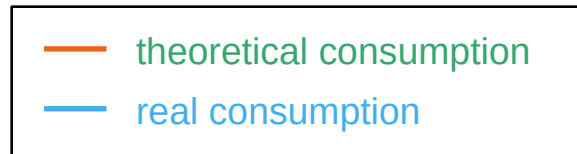
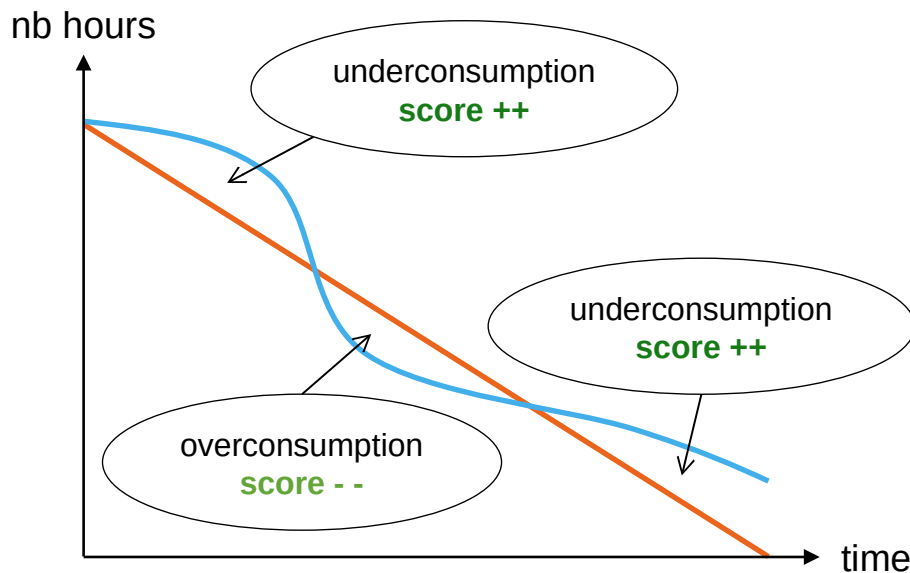


Job submission with Slurm



Job submission with Slurm

Slurm gives your job a **priority score** depending on your consumption.



Slurm compares all users' scores to define job position in the queue.

Job submission with Slurm

Adjust the **QoS (Quality of Service)** to improve the priority score of your job!



QoS	Max elapsed time	Resource limits			
		Per job	Per user	Per project	Per QoS
qos_gpu_h100-dev	2h	32 GPU	32 GPUs (10 jobs max at the same time)	32 GPU	384 GPU
qos_gpu-t3 (default)	20h	512 GPU	512 GPU	512 GPU	
qos_gpu_h100-t4	100h	16 GPU	64 GPU	64 GPU	192 GPU



dev

prod

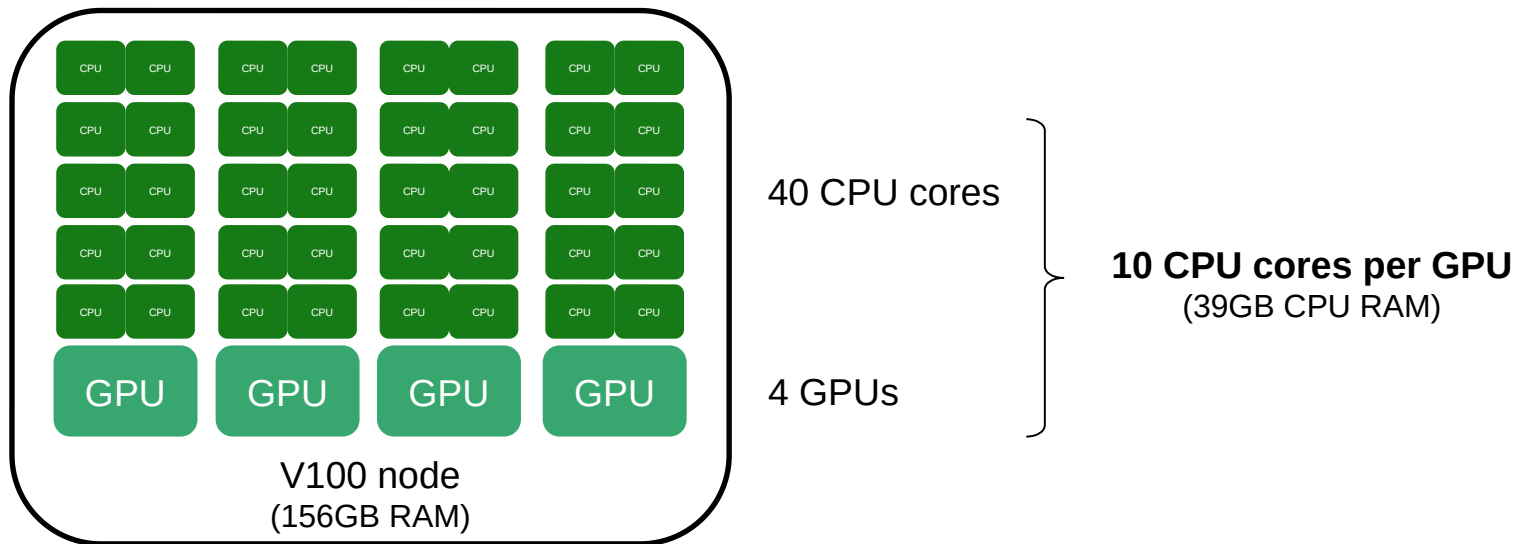
And keep your job in the queue, its priority score will increase with time.

Job submission with Slurm

How to configure my job?

- How many GPUs?
- How much time?
- How many CPUs per GPU?

→ **1 CPU core = 3.9GB RAM** (on V100 nodes)
→ interesting to have **several CPU cores to feed a GPU**
(cf “DataLoader optimizations” part of this course)

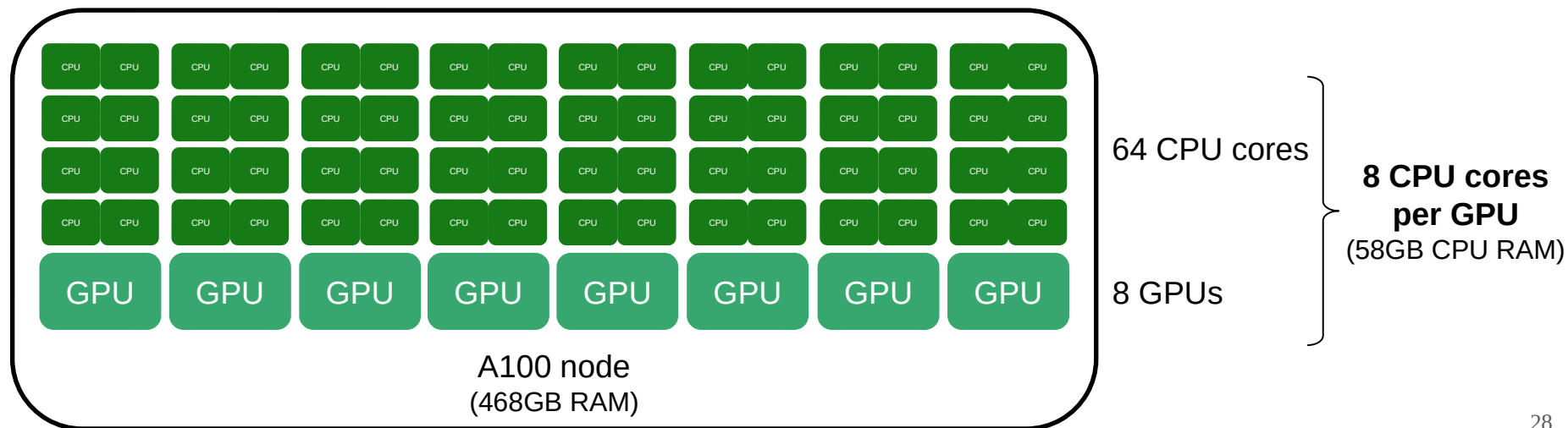


Job submission with Slurm

How to configure my job?

- How many GPUs?
- How many time?
- How many CPUs per GPU?

→ **1 CPU core = 7.3GB RAM** (on A100 nodes)
→ interesting to have **several CPU cores to feed a GPU**
(cf “DataLoader optimizations” part of this course)

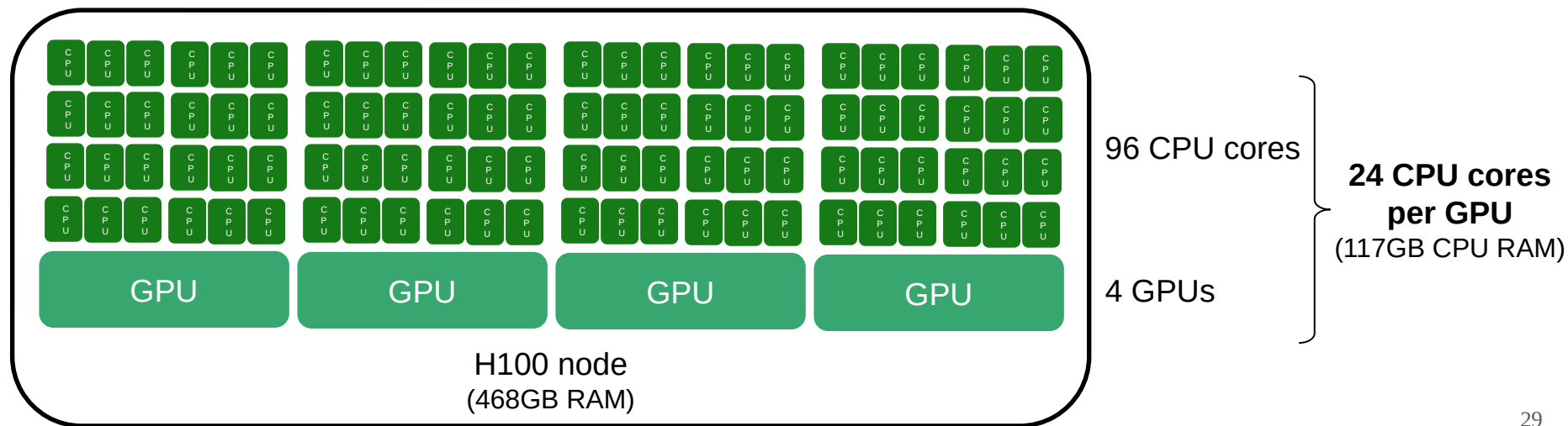


Job submission with Slurm

How to configure my job?

- How many GPUs?
- How many time?
- How many CPUs per GPU?

→ **1 CPU core = 4.87GB RAM** (on H100 nodes)
→ interesting to have **several CPU cores to feed a GPU**
(cf “DataLoader optimizations” part of this course)



Job submission with Slurm

Example: reservation of 2 x 4 H100 GPUs

script.slurm

```
#!/bin/bash

#SBATCH --job-name="dlojz"           # name of the job
#SBATCH --output="dlojz%j.out"       # output file
#SBATCH --error="dlojz%j.err"        # error file
#SBATCH --nodes=2                    # nb of nodes
#SBATCH --gres=gpu:4                 # nb of GPUs/node
#SBATCH --ntasks-per-node=4          # nb of tasks/node
#SBATCH --cpus-per-task=24           # nb of CPU cores/task
#SBATCH --hint=nomultithread         # no hyperthreading
#SBATCH --time=01:00:00              # max execution time
#SBATCH --qos=qos_gpu_h100-dev       # adjust QoS

module load pytorch-gpu/py3/2.8.0    # environment

srun python script.py                 # run script
```

Job submission with Slurm

script.slurm

```
#!/bin/bash
```

```
#SBATCH --job-name="dlojz"
```

```
#SBATCH --output="dlojz%j.out"
```

```
#SBATCH --error="dlojz%j.err"
```

```
#SBATCH --nodes=2
```

```
#SBATCH --gres=gpu:4
```

```
#SBATCH --ntasks-per-node=4
```

```
#SBATCH --cpus-per-task=10
```

```
#SBATCH --hint=nomultithread
```

```
#SBATCH --time=01:00:00
```

```
#SBATCH --qos=qos_gpu-dev
```

```
module load pytorch-gpu/py3/2.8.0
```

```
srun python script.py
```

```
login@jean-zay3:~$ sbatch script.slurm
```

Job submission



Waiting in queue

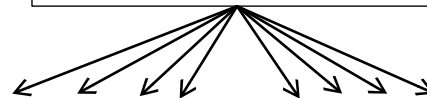
```
login@jean-zay3:~$ squeue --me
```

JOBID	PARTITION	NAME	USER	ST	PD	TIME	NODES	ODELIST(Reason)
223225	gpu_p13	dlojz				0:00	2	(Priority)



Launching job

```
srun python script.py
```



NODE 0

GPU0

GPU1

GPU2

GPU3

NODE 1

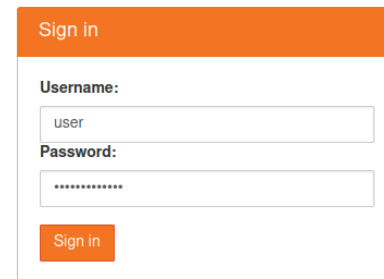
GPU0

GPU1

GPU2

GPU3

1. Authentication on <https://jupyterhub.idris.fr>



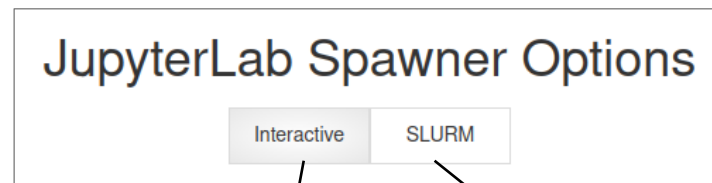
Sign in

Username:

Password:

Sign in

2. Choose and configure an instance



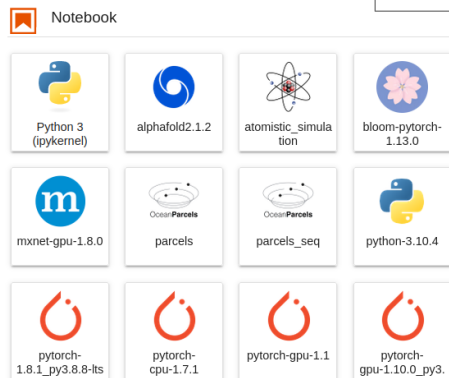
JupyterLab Spawner Options

Interactive SLURM

Run on a
connection node

Run on a
compute node

3. Choose a kernel
(pytorch-gpu-2.2.0)



Slurm tools for python notebooks

```
from idr_pytools import gpu_jobs_submitter
```

```
command = 'dlojz.py --batch-size 128 --image_size 176'
```

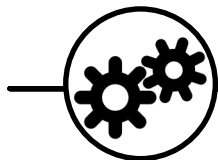
```
n_gpu = 8
```

```
MODULE = 'pytorch-gpu/py3/2.2.0'
```

```
name = 'dlojz'
```

```
jobid = gpu_jobs_submitter(command, n_gpu, MODULE, name=name, account='xyz@a100', time_max='05:00:00')
```

command
n_gpu
MODULE
name
account
time_max



script.slurm

```
#!/bin/bash

#SBATCH --job-name="dlojz"
#SBATCH --output="dlojz%j.out"
#SBATCH --error="dlojz%j.err"
#SBATCH --nodes=2
#SBATCH --gres=gpu:8
#SBATCH -C a100
#SBATCH --ntasks-per-node=8
#SBATCH --cpus-per-task=8
#SBATCH --hint=nomultithread
#SBATCH --time=05:00:00
#SBATCH --account=xyz@a100

module load pytorch-gpu/py3/2.1.1

srun python dlojz.py --batch-size 128 --image_size 176
```

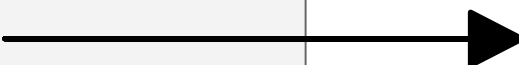
```
$ sbatch script.slurm
```

jobid

Slurm tools for python notebooks

```
from idr_pytools import display_slurm_queue
```

```
name = 'dlojz'  
display_slurm_queue(name)
```



\$ squeue --me -n <name>

```
from idr_pytools import search_log
```

```
jobid = ['12345']
```

```
search_log(contains=jobid)[0]
```



output filename

```
search_log(contains=jobid, with_err=True)[0]
```



error filename

The Challenges of Scaling

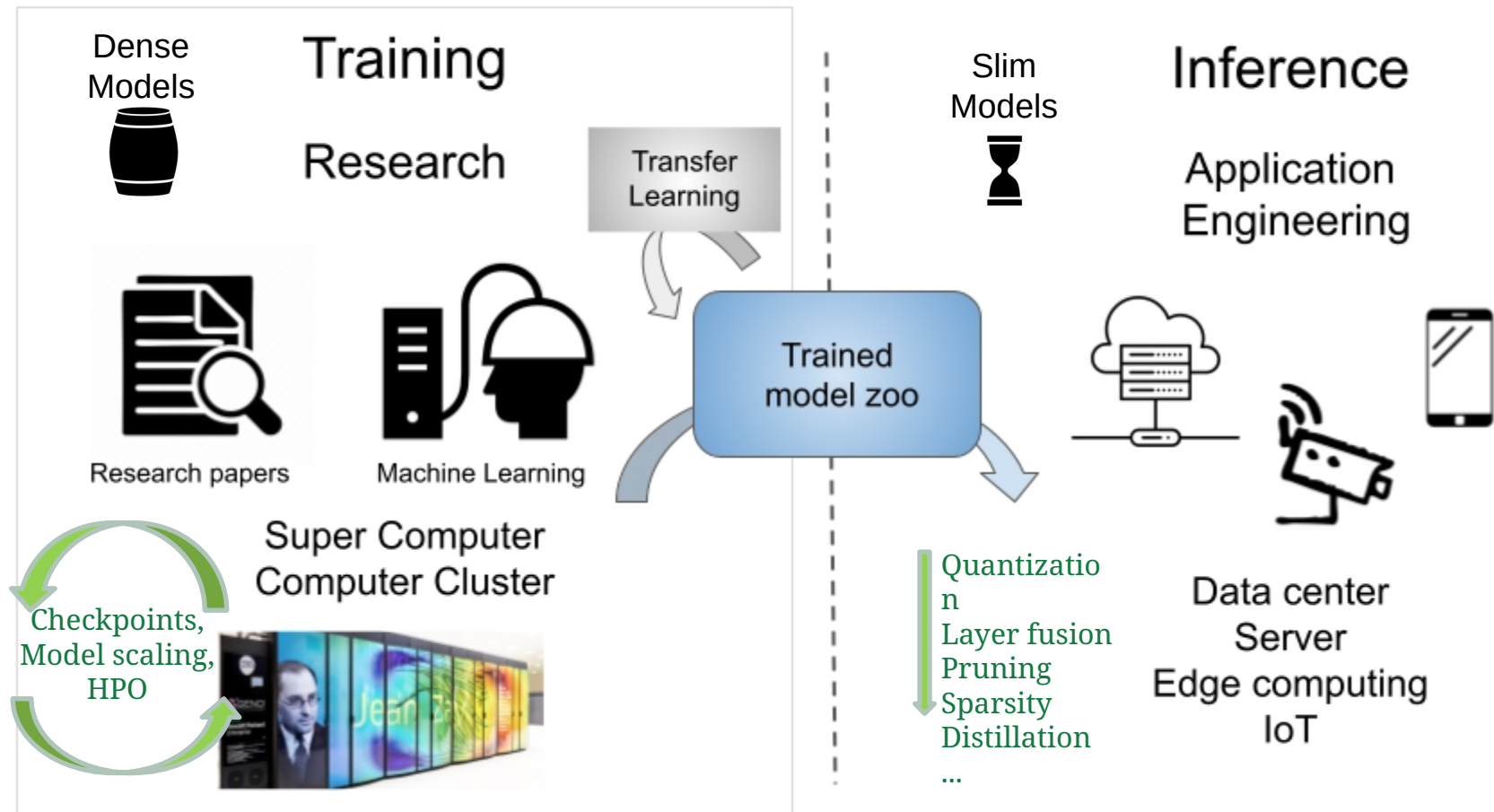
Training Time ◀

Memory Footprint ◀

Solutions ◀

Energy saving ◀

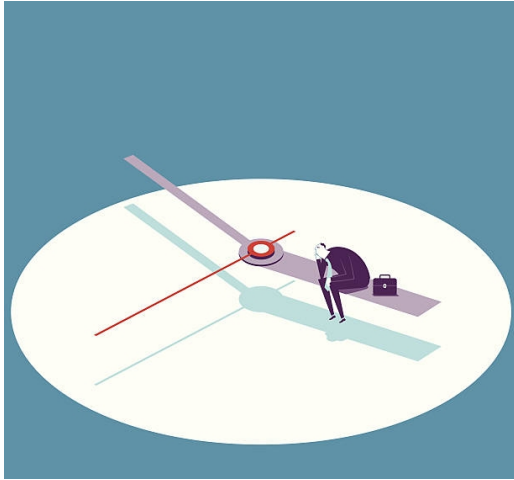
Training / Inference



Constraints of Deep Learning

2 problems to deal with:

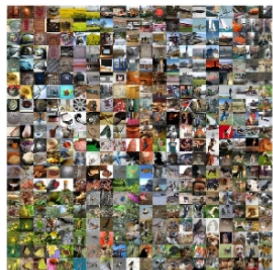
Training Time



Memory Overconsumption (OOM)



Training Resnet-50 on Imagenet



Goal:

Classification (1000 classes)

Model :

25 M parameters

Dataset:

1,2 M labeled images

14 Days

1 NVIDIA M40 GPU

Training Time

Training Resnet-50 on Imagenet

Facebook Caffe2	UC Berkeley, TACC, UC Davis Tensorflow	Preferred Network ChainerMN	Tencent TensorFlow	Sony Neural Network Library (NNL)	Fujitsu MXNet
1 hour	31 mins	15 mins	6.6 mins	2.0 mins	1.2 mins
Tesla P100 x 256	1,600 CPUs	Tesla P100 x 1,024	Tesla P40 x 2,048	Tesla V100 x 3,456	Tesla V100 x 2,048
Apr	Sept	Nov	July	Nov	Apr
2017				2018	2019

Fast.ai tips and engineering

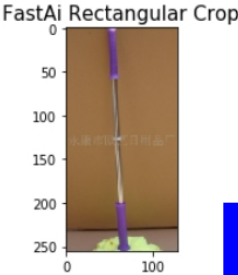


« Now anyone can train Imagenet in 18 minutes »

Our approach uses **128** processing units and costs around **\$40** to run.

👏 OneCycle lr scheduler
+ lr finder

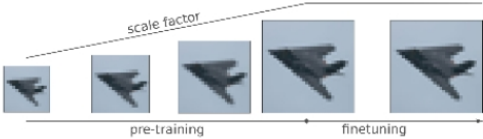
➡ Popularizes the works of
[Leslie N. Smith](#)



Thanks to Global Average Pooling

Test Rectangular
Validation Technique

Progressive image
resizing



Dynamic batch size



Training Time



Goal:

Text Generation Foundation Model (LLM)

Model :

176 B parameters

Dataset:

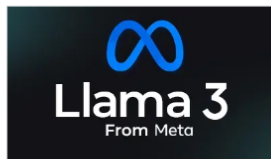
366 B tokens



117 Days
384 A100 GPUs



Training Time



Goal:

Text Generation Foundation Model (LLM)

Model :

405 B parameters

Dataset:

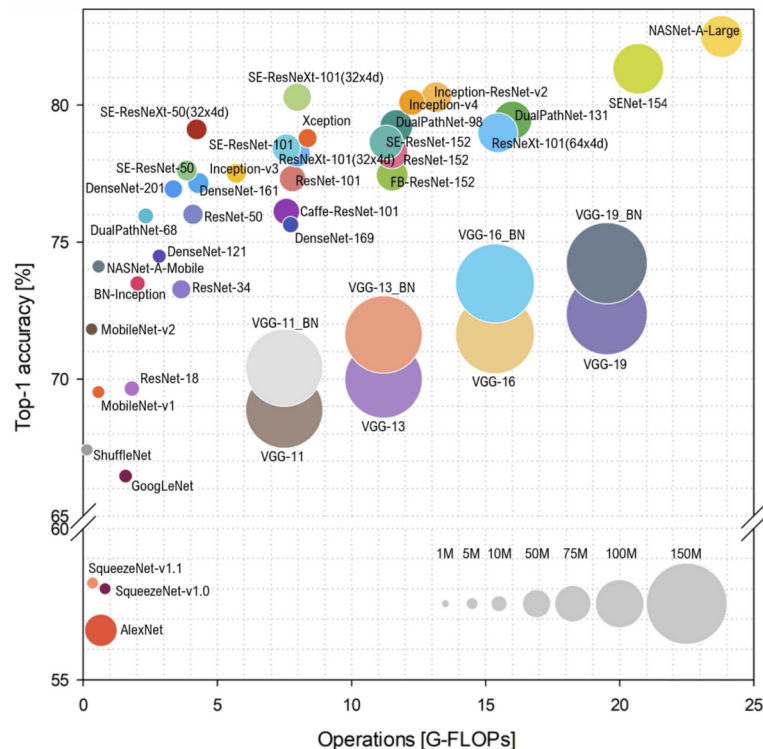
15 T tokens

54 Days

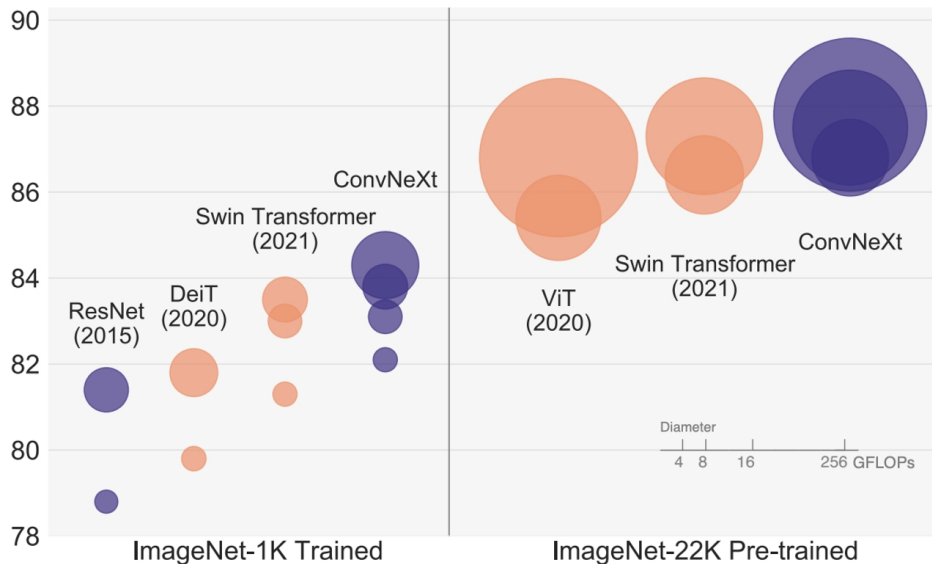
16 384 H100 GPUs

Large Model

Vision Neural Network



ImageNet-1K Acc.

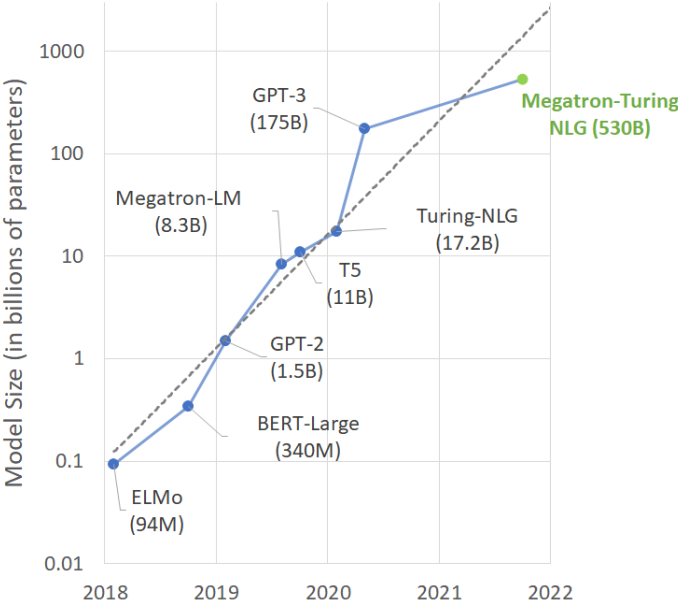


Large and deep models provide better accuracy metrics.

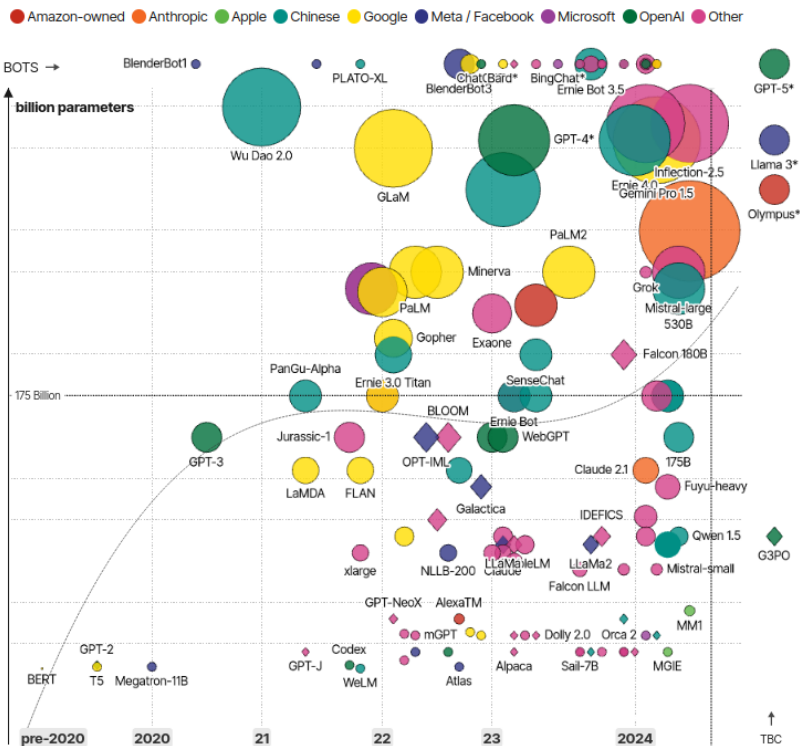
Large Model

Huge models cause very expensive compute work times and large memory footprints (4 GB for a model with 1 billion parameters).

Transformers



The Rise and Rise of A.I. Large Language Models (LLMs) & their associated bots like ChatGPT

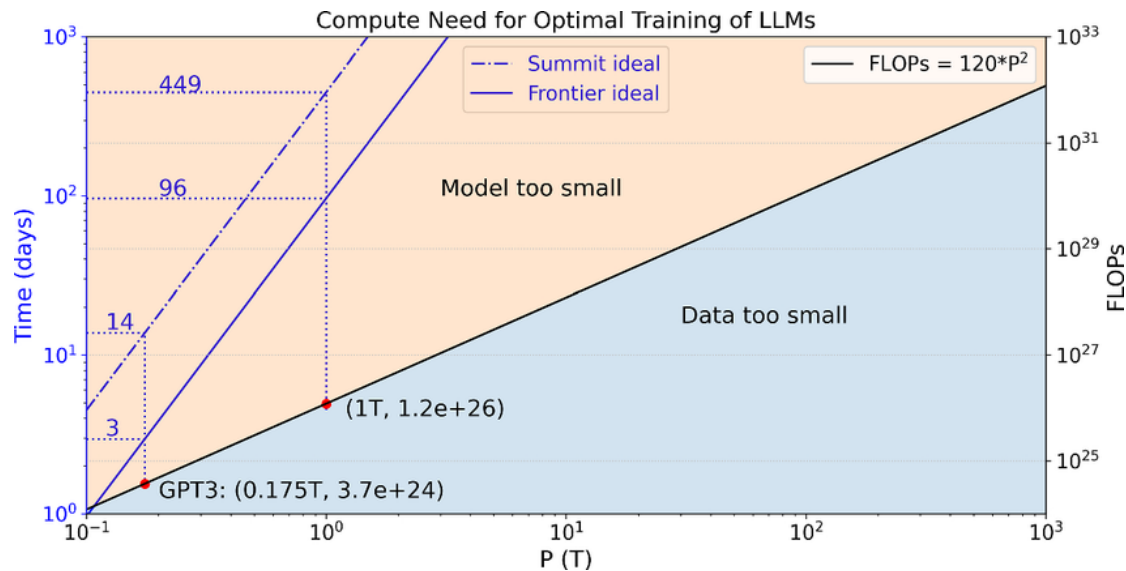
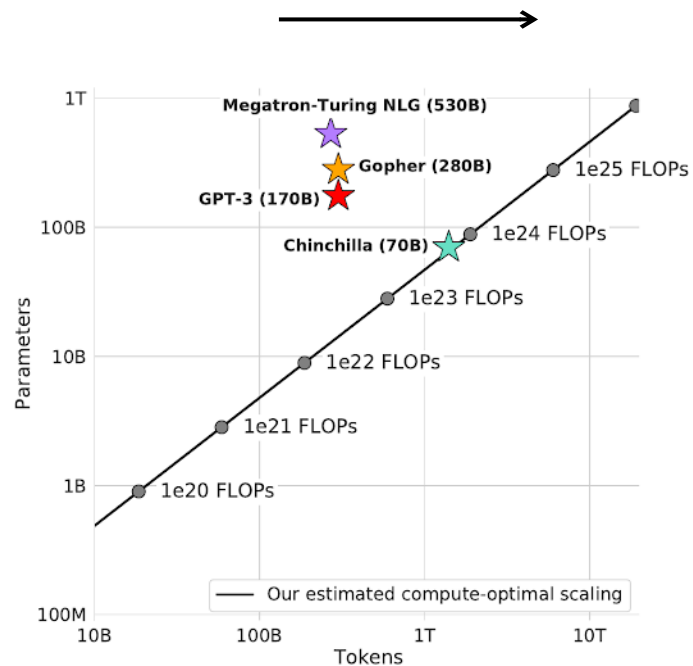


David McCandless, Tom Evans, Paul Barton
Information is Beautiful // UPDATED 20th Mar 24
source: news reports, LifeArchitecture
* = parameters undisclosed // see the data

MADE WITH VIZsweat

Large Model

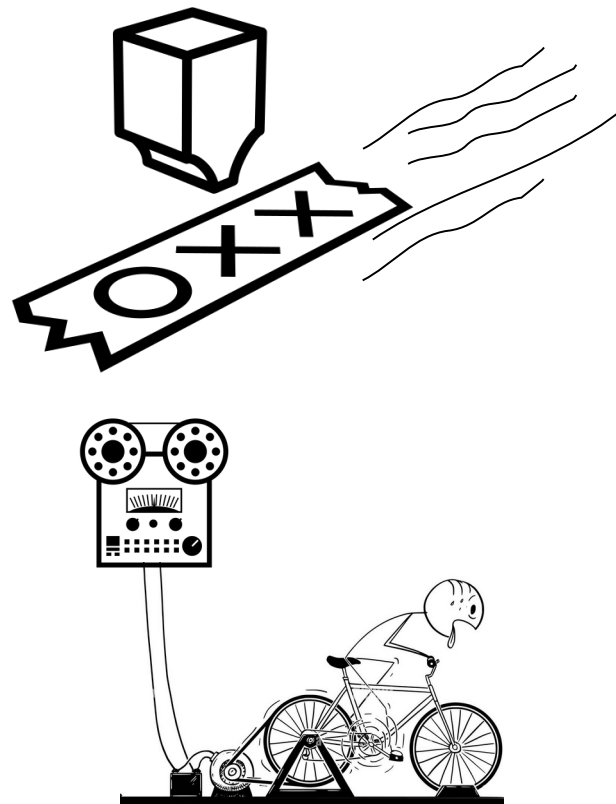
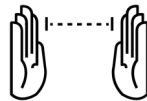
Chinchilla Law Impact



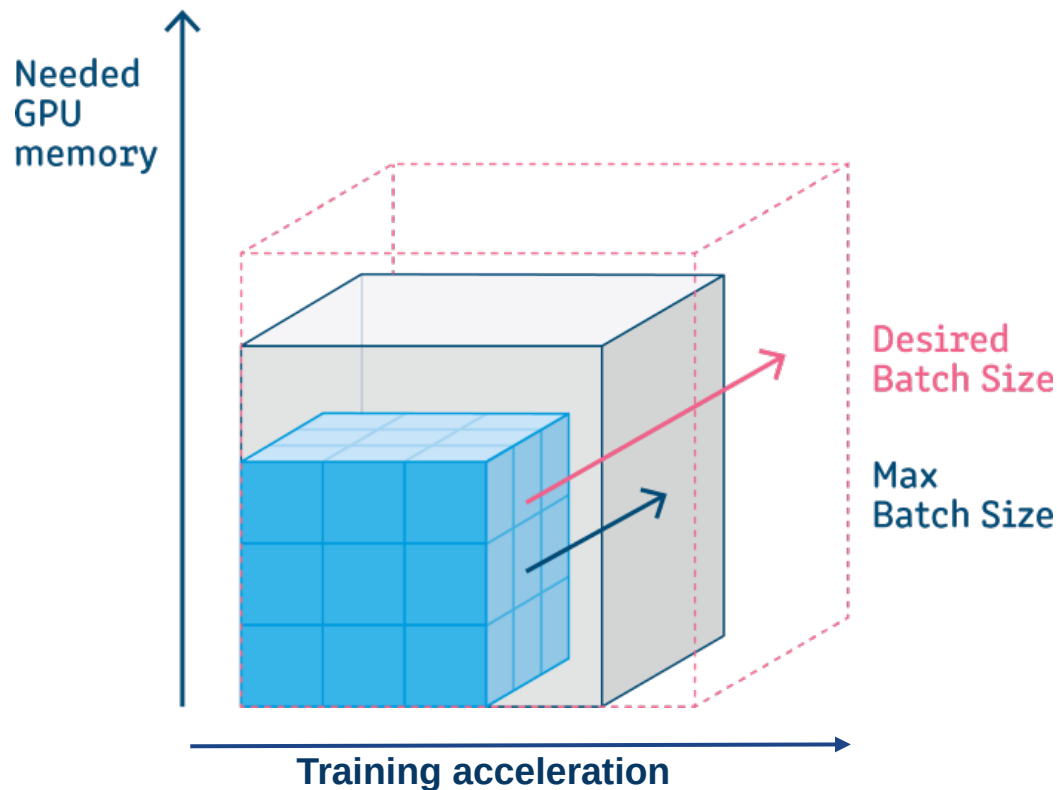
Compute Work Times

The compute time increases with the **number of FLOPs** required, depending on:

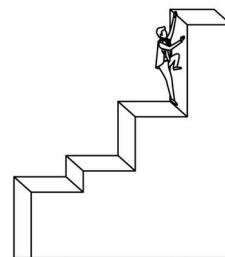
- The size of the model
- The depth of the model
- The size of the input data (image resolution, length of the sequence, etc.)
- The size of the dataset
- Number of epochs required



Batch Size & Memory Usage



Increasing the batch size and thus increasing the iteration step speeds up learning.



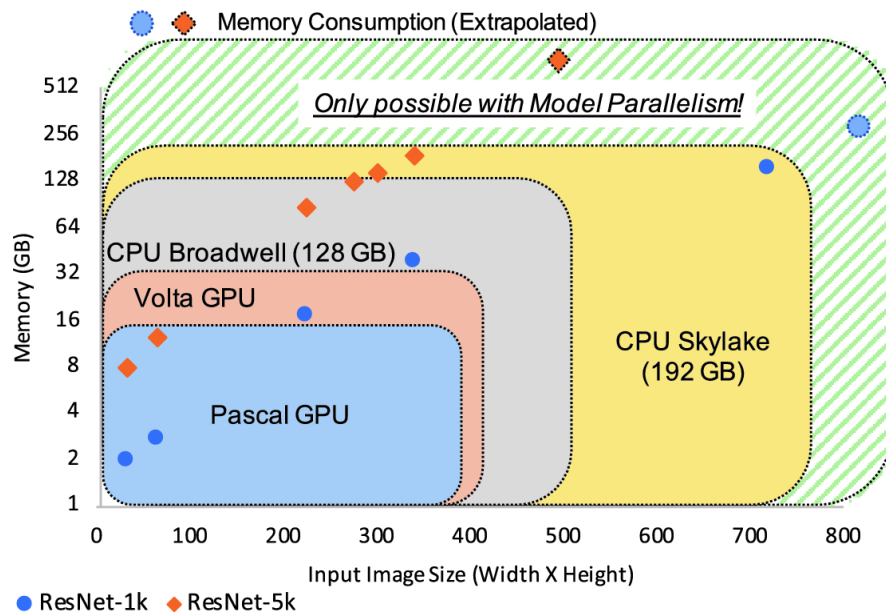
OOM
Process killed

However, this increases the memory footprint and risks reaching the system limit.

Large size Input Data

Large input data causes **serious memory occupancy problems** during training, **accentuated by the depth of the model**.

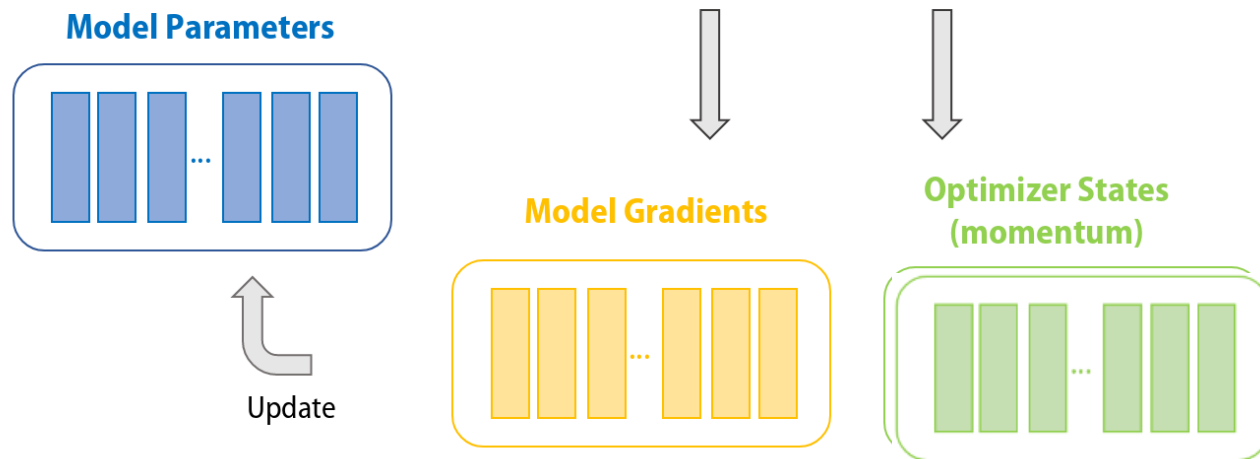
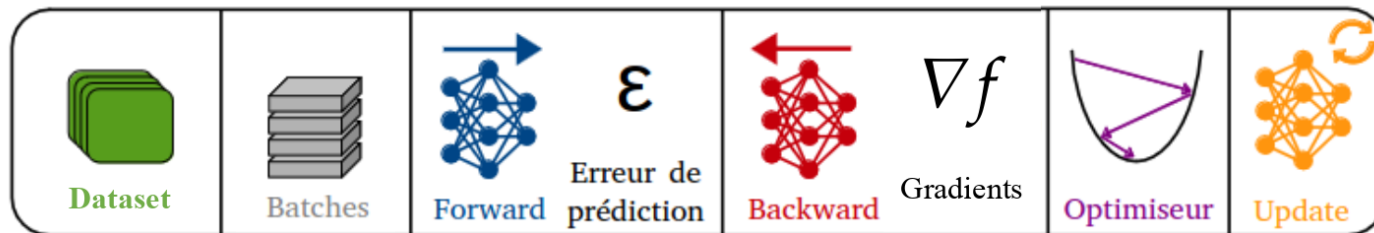
- Text (N, 100, 500) ~x1
- Image 2D (N, 226, 226, 3) ~x3
- Image 3D (N, 226, 226, 100, 3) ~x300
- Video (N, 100, 226, 226, 3) ~x300



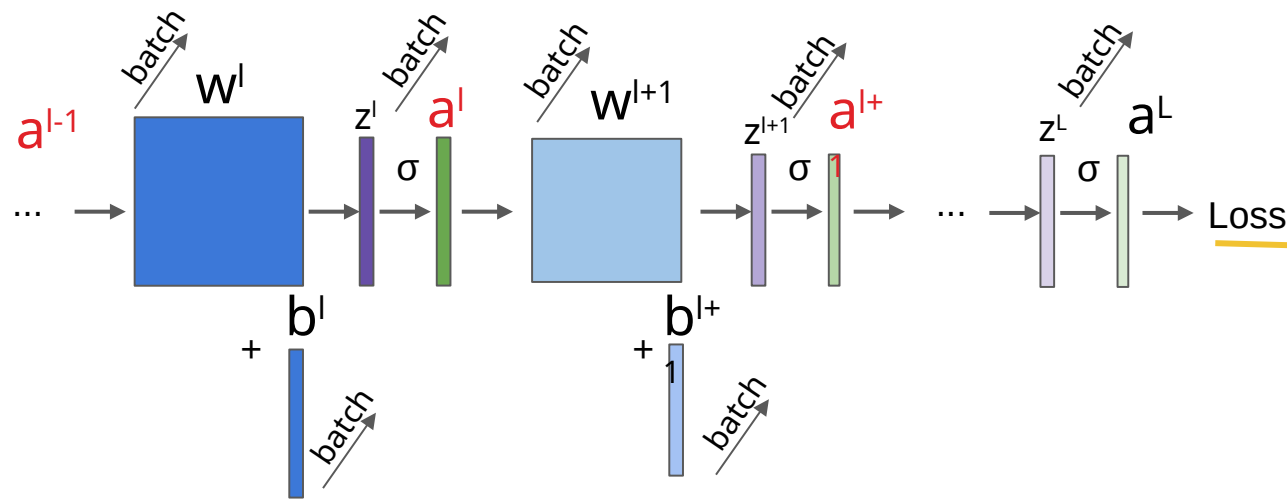
(GNN : Graph from tiny to huge !!)

Source : [HyPar-Flow](#)

Forward / Backward – Model Memory Occupancy



Forward / Backward – Activations Memory Occupancy



Propagation

$$a^l = \sigma(w^l a^{l-1} + b^l) = \sigma z^l$$

Backpropagation

$$\delta^l = \frac{\partial C}{\partial z^l} \quad \begin{aligned} w^l &\rightarrow w^l - \frac{\eta}{m} \cdot \frac{\partial C}{\partial w^l} \\ b^l &\rightarrow b^l - \frac{\eta}{m} \cdot \frac{\partial C}{\partial b^l} \end{aligned}$$

$$\delta^L = \nabla_a C \odot \sigma'(z^L)$$

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l)$$

$$\frac{\partial C}{\partial w^l} = \delta^l (a^{l-1})^T$$

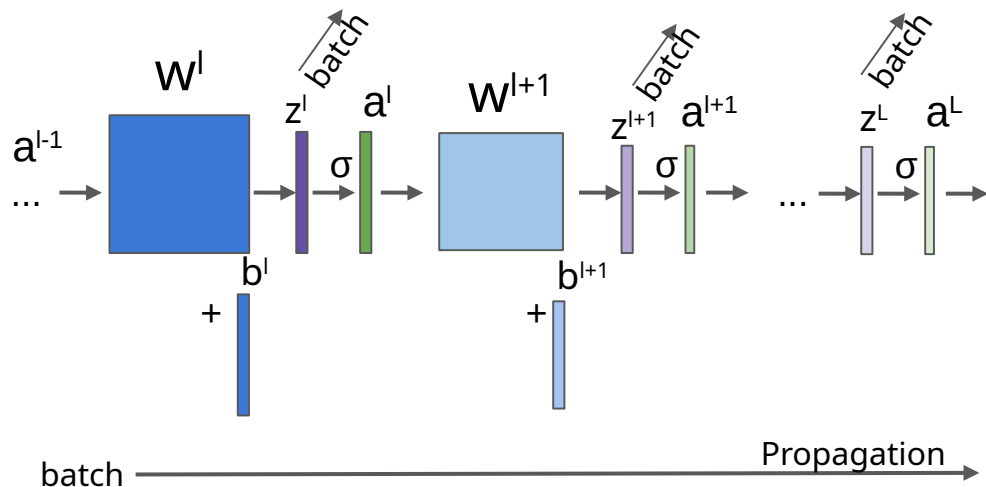
$$\frac{\partial C}{\partial b^l} = \delta^l$$

batch $\xrightarrow{\text{Propagation n}}$ batch

averaged gradients $\xleftarrow{\text{Backpropagation n}}$

Note: For backpropagation, it is necessary to save **intermediate activations**.

Inference & evaluation



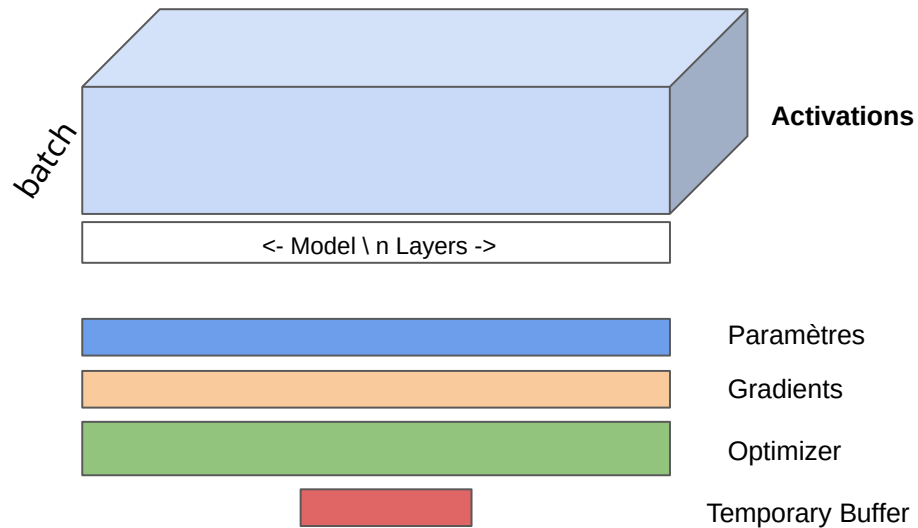
Propagation

$$a^l = \sigma(w^l a^{l-1} + b^l) = \sigma z^l$$

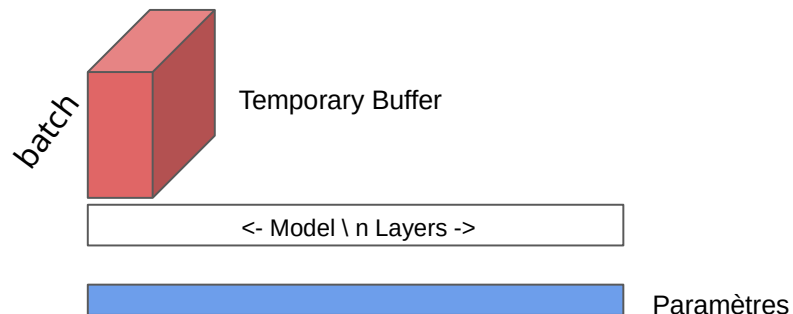
```
...  
with torch.no_grad():  
    val_outputs = model(val_images)  
    loss = criterion(val_outputs, val_labels)  
...
```

Memory Footprint

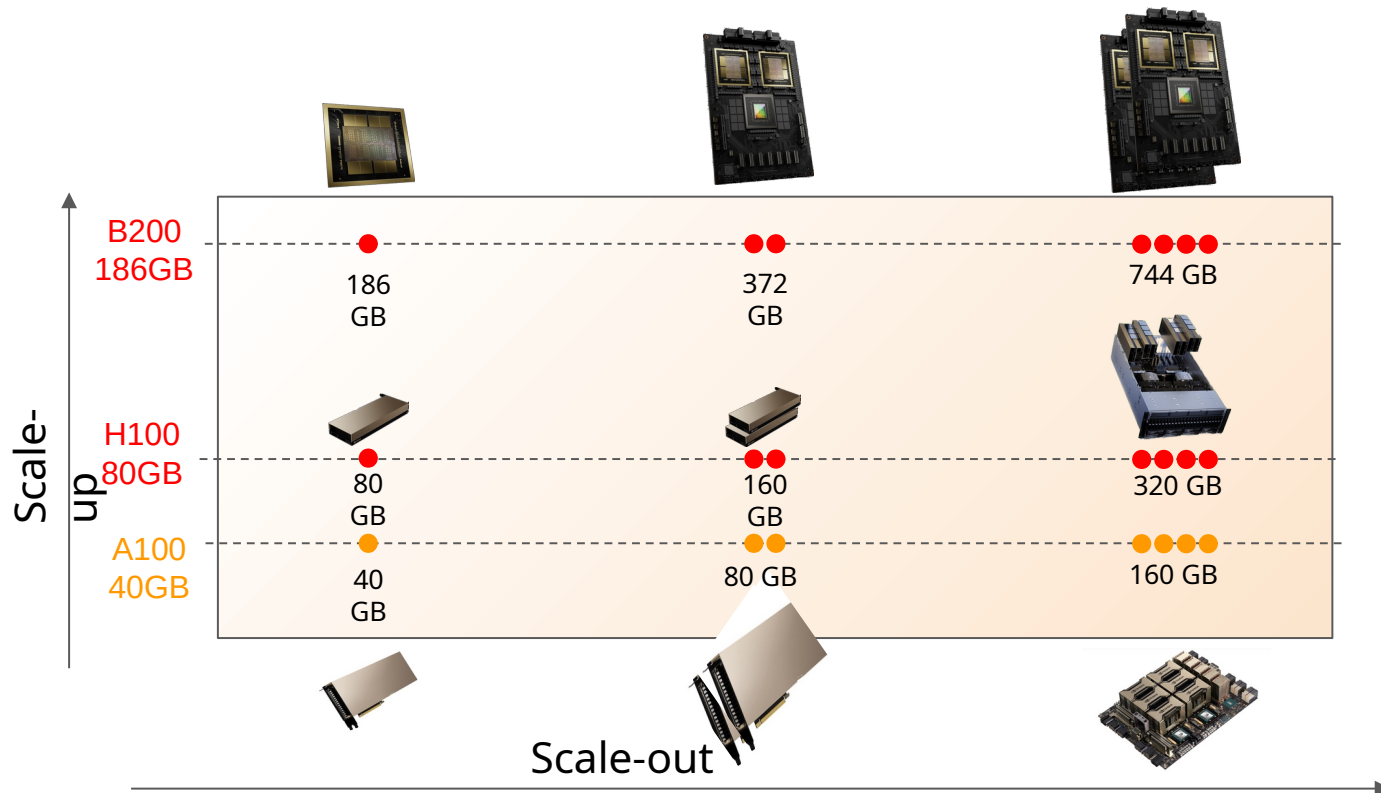
Training



Inference / Evaluation

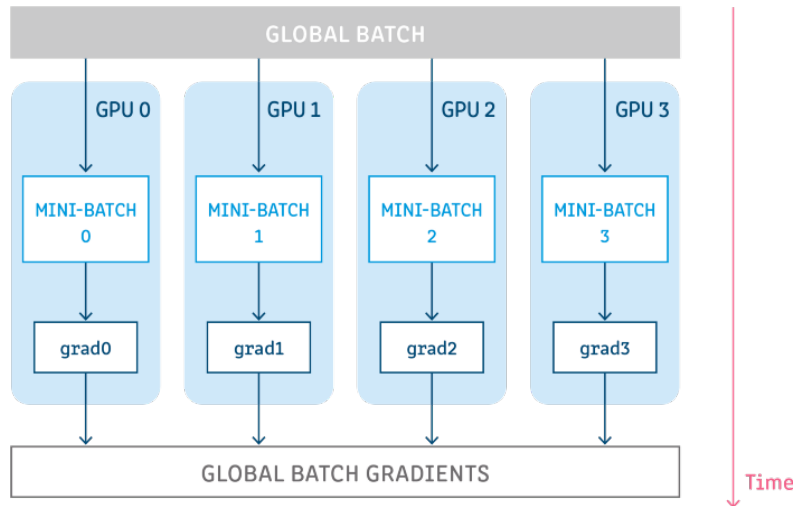


System Solutions

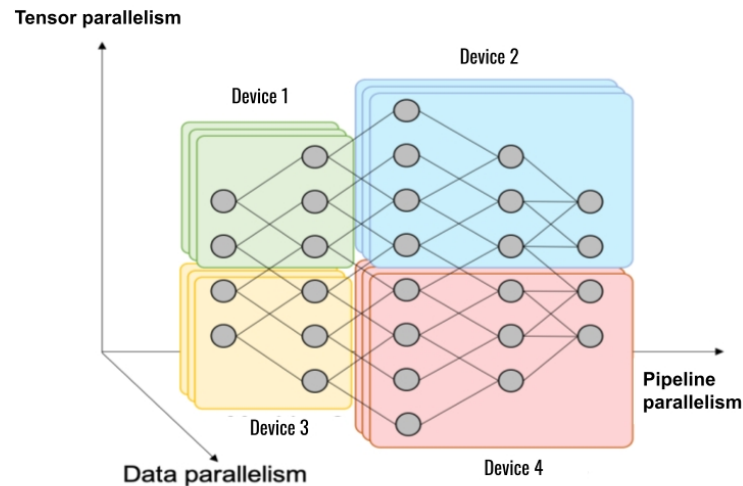


Solutions: Distribution – Scale-out

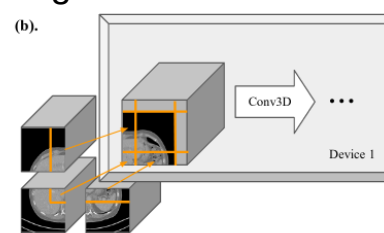
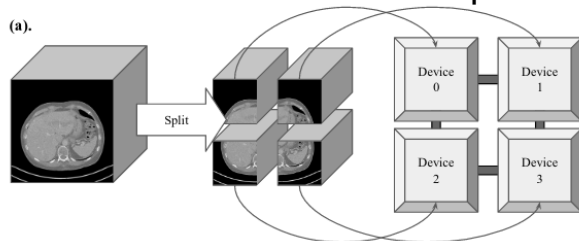
Data Parallelism



Model Parallelism (Tensor + Pipeline)

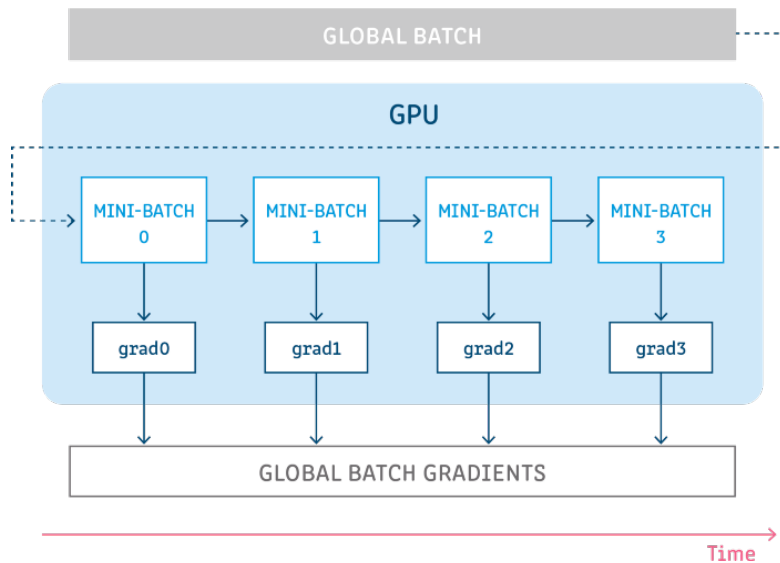


Spatial Partitioning



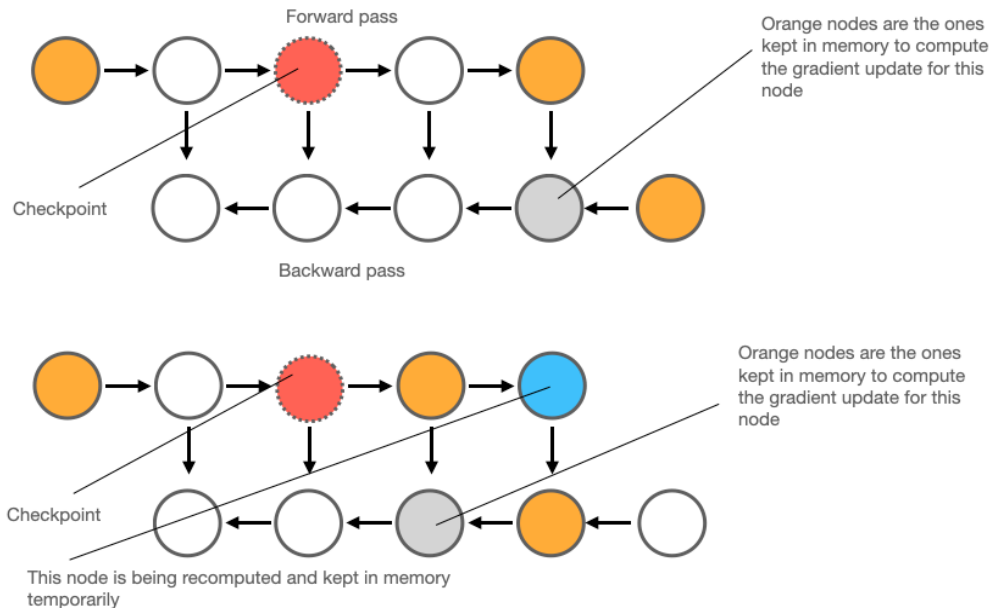
Workaround Solutions

Gradient aggregation



Gradient/activation checkpointing

Drawing inspired by <https://github.com/cybertronai/gradient-checkpointing>



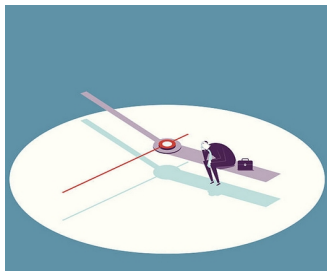
A 3rd problem to deal with ...

Power Consumption !!

2 problems to deal with:

Training Time

Memory Overconsumption (OOM)



Power Consumption

	H100 PCIe	H100 SXM5	A100 PCIe	A100 SXM4	V100 PCIe	V100 SXM2
Max Power	350W	700W	250W	400W	250W	300W
Idle Power	~30W	~55W	~30W	~60W	~40W	~45W
Performance	x3.5	x4	x1.75	x2	x0.9	x1

For a node: The CPU (often 2 processors) consumes what approximately 1 GPU consumes.



Power consumption varies depending on partial or overall GPU usage.

However, the power efficiency ratio is in favor of full use of the GPU.

Energy Consumption / GPU Hours

Energy Saving

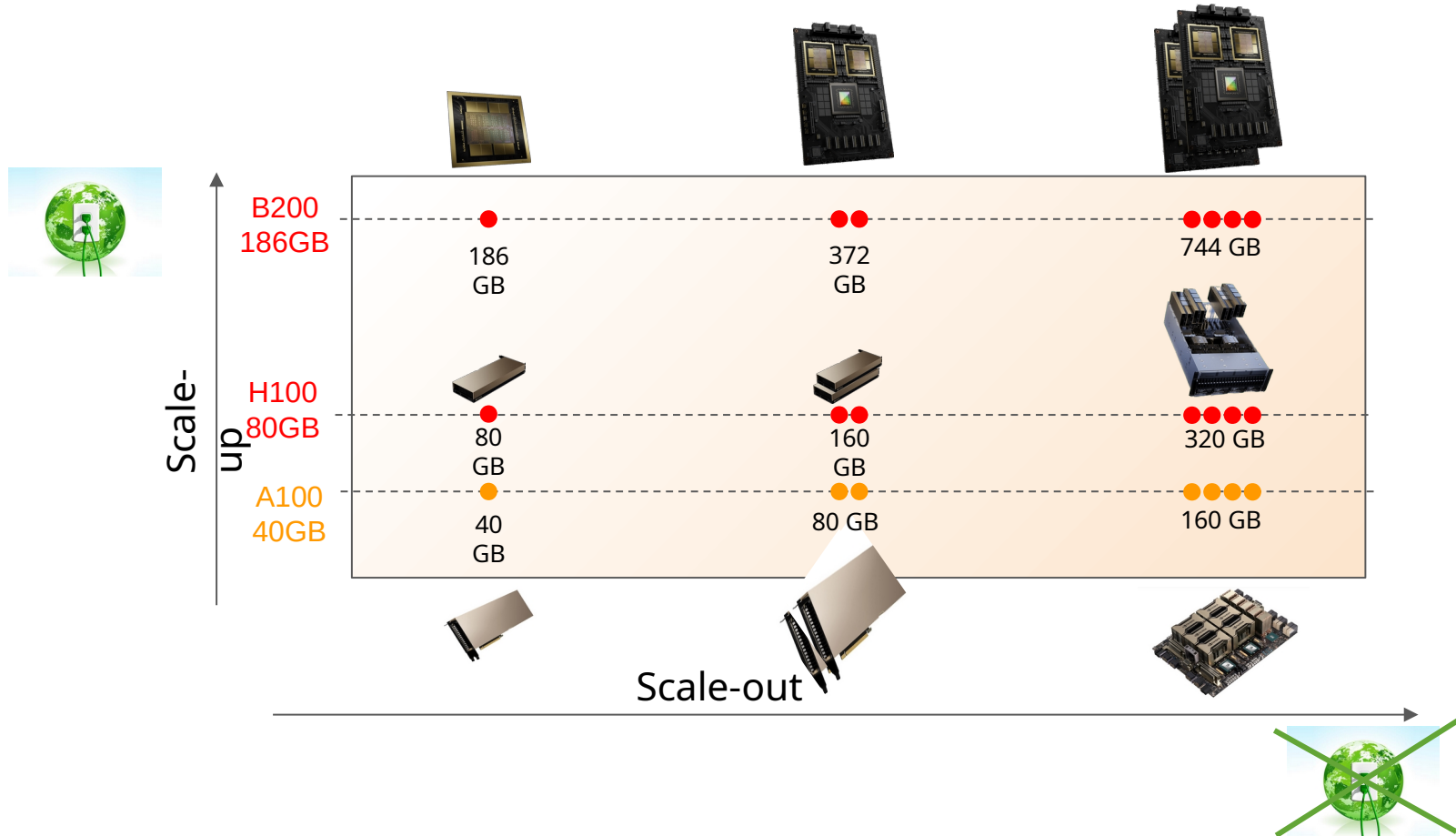
GPU Hours Saving



System optimization (DLO-JZ)

- Find the most important throughput
- Optimize data loading to eliminate GPU idle times
- Parallelize training to the right scale: neither too much nor too little

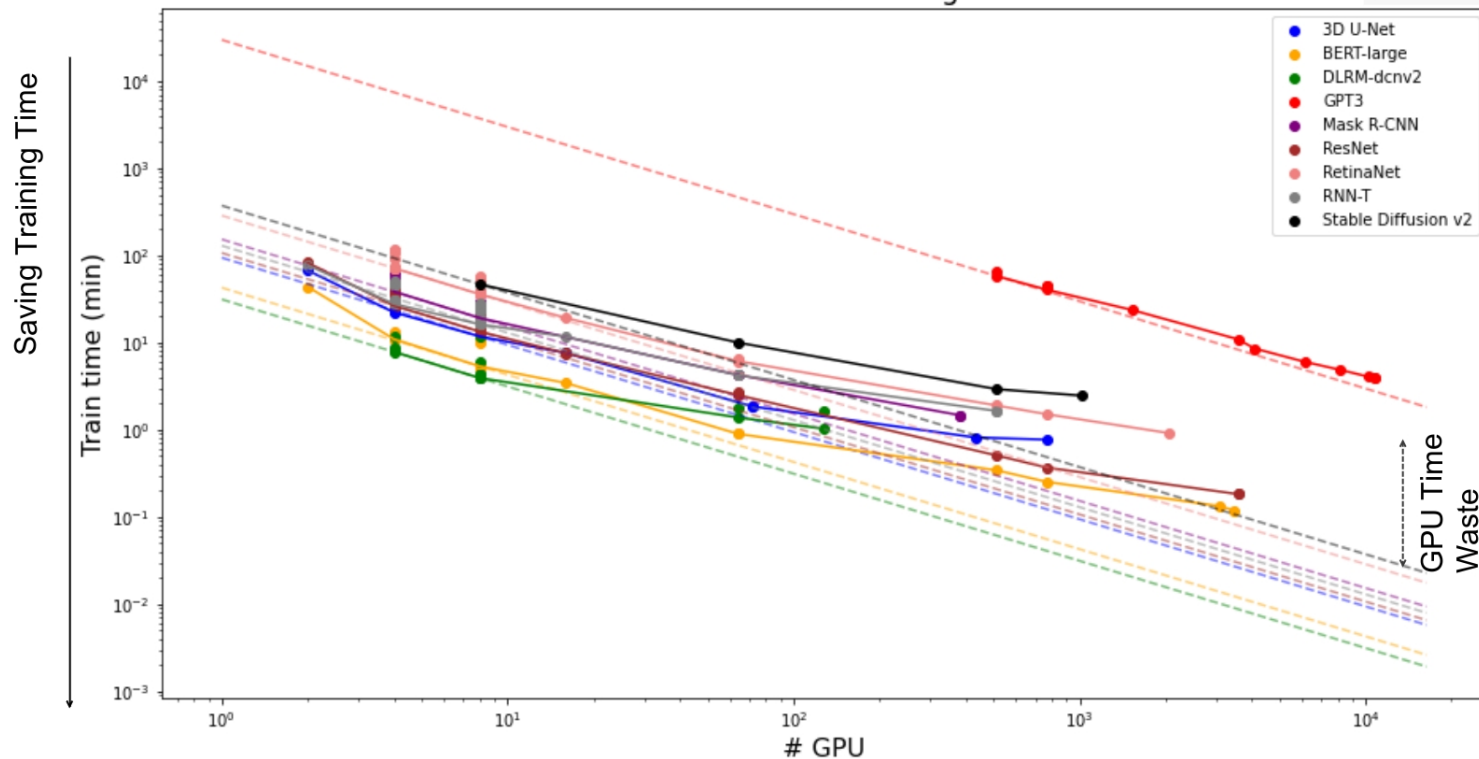
Energy Consumption / GPU Hours



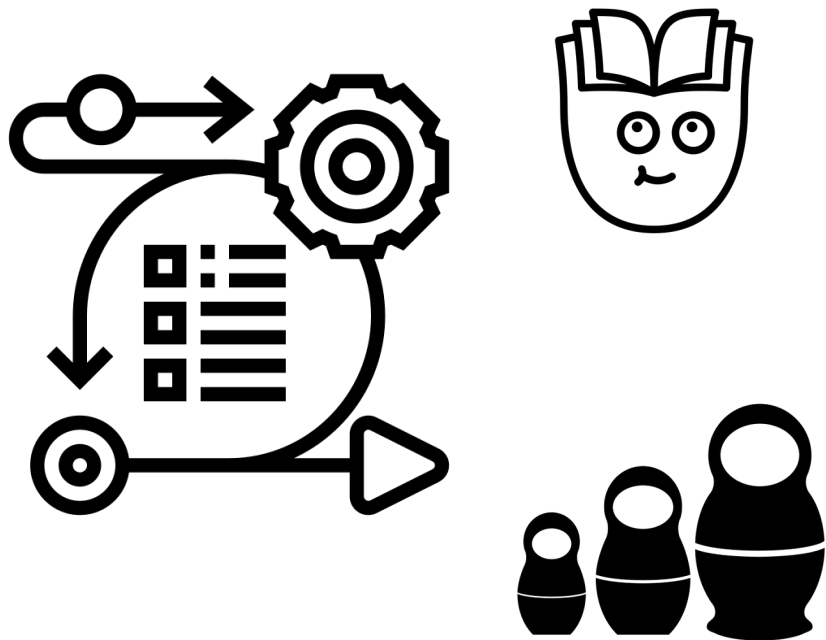
ML Perf Result - Scaling



MLPerf - H100 - Training v3.0



Energy Consumption / GPU Hours



Methodology (saving research, not repeating learning unnecessarily)

- Search for hyperparameters in publications and reproduce the state-of-the-art
- Find the right hyper parameters on smaller models, then apply them at scale
- *Hyper-Parameter Optimization* (HPO) techniques

Code review

General overview ◀

Detailed overview ◀

Data - Imagenet

Goal:

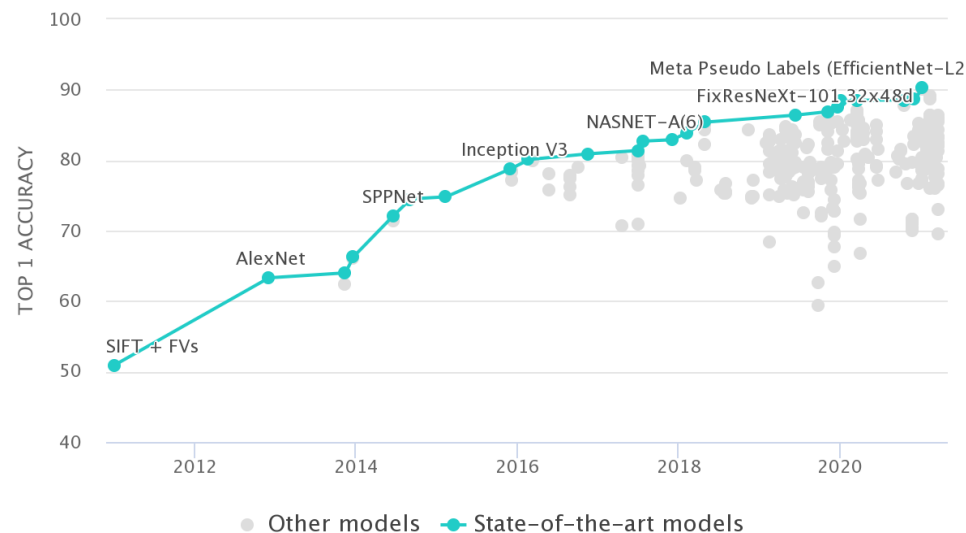
Classification (1000 classes)

Dataset:

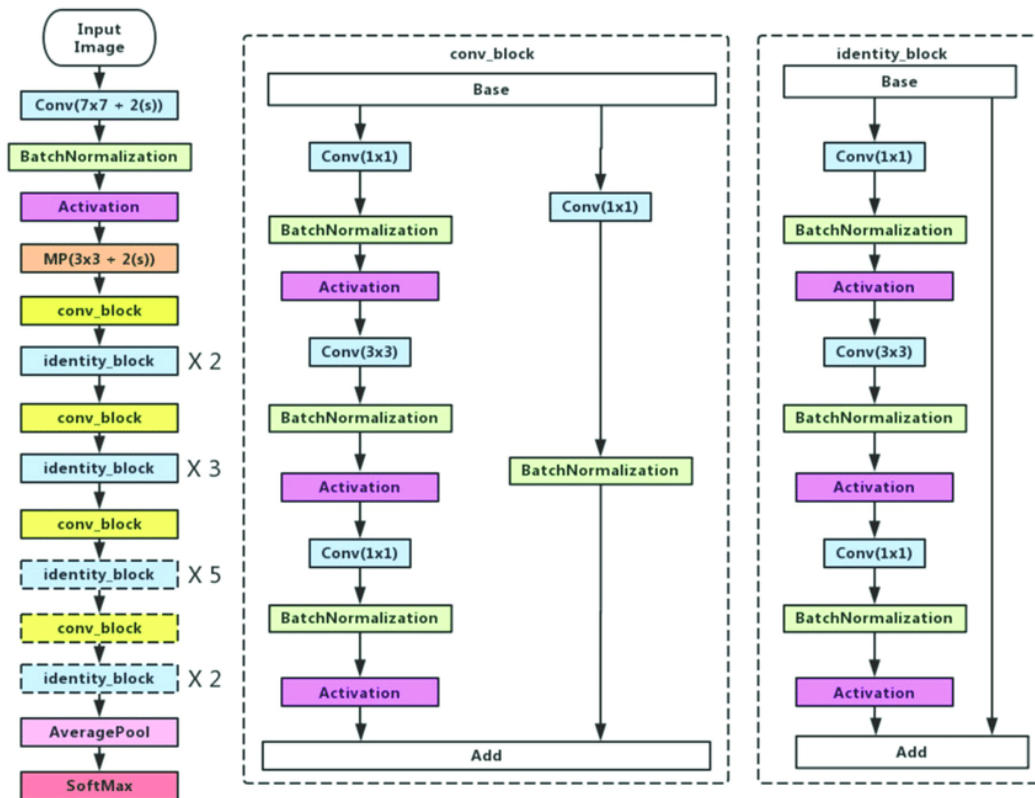
Train dataset: **1,2 M** labeled images

Validation dataset: **50 000** labeled images

<http://www.image-net.org/>



Imagenet - Resnet



Resnet :

- Residual Learning
- BatchNorm layer
 - Instead of Bias layers with conv.
- Average Pooling
 - Makes the model independent of the size of the input images

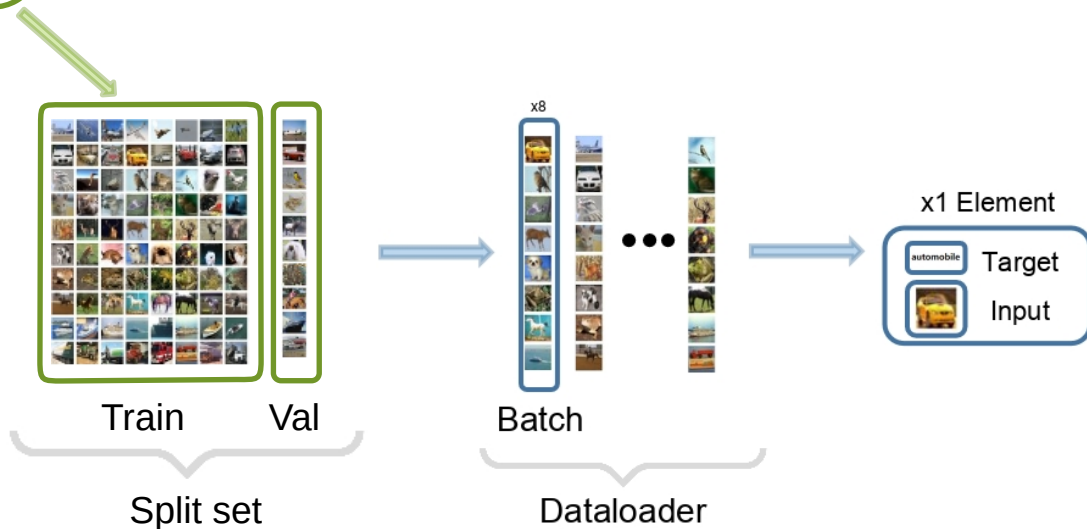
Training Loop – DataLoader

Dataset

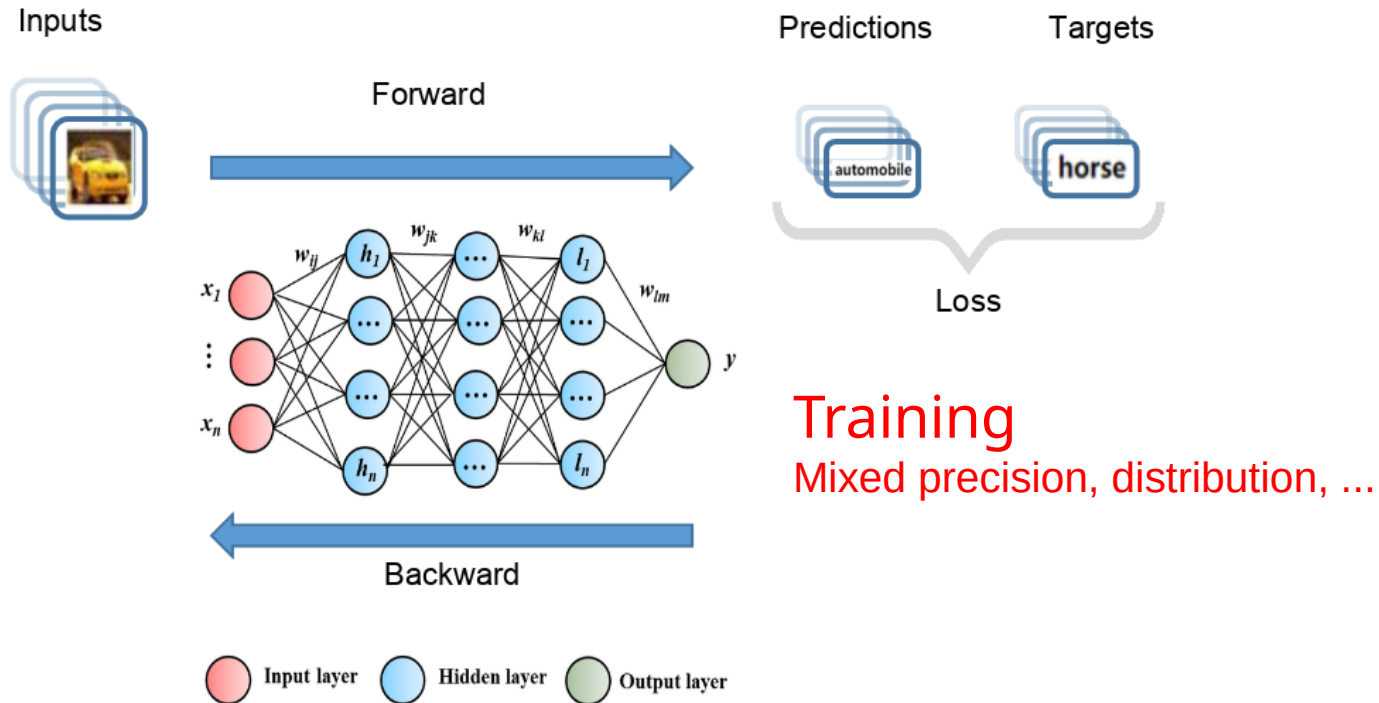


Dataloader

Preprocessing, Data Augmentation, Batching ...



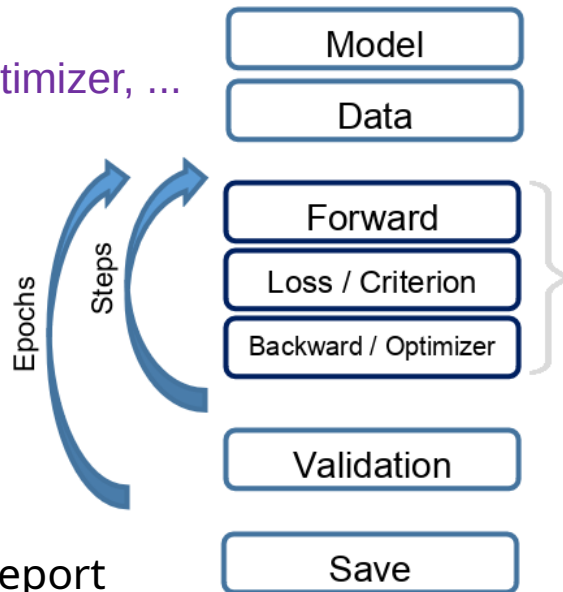
Training Loop – Forward/Backward



Training Loop

Instanciación

Model, distribution, optimizer, ...



Dataloader

Preprocessing, Data Augmentation, Batching ...

Training

Mixed precision, distribution, ...

Validation

Mixed precision, distribution, ...

Checkpoint & report

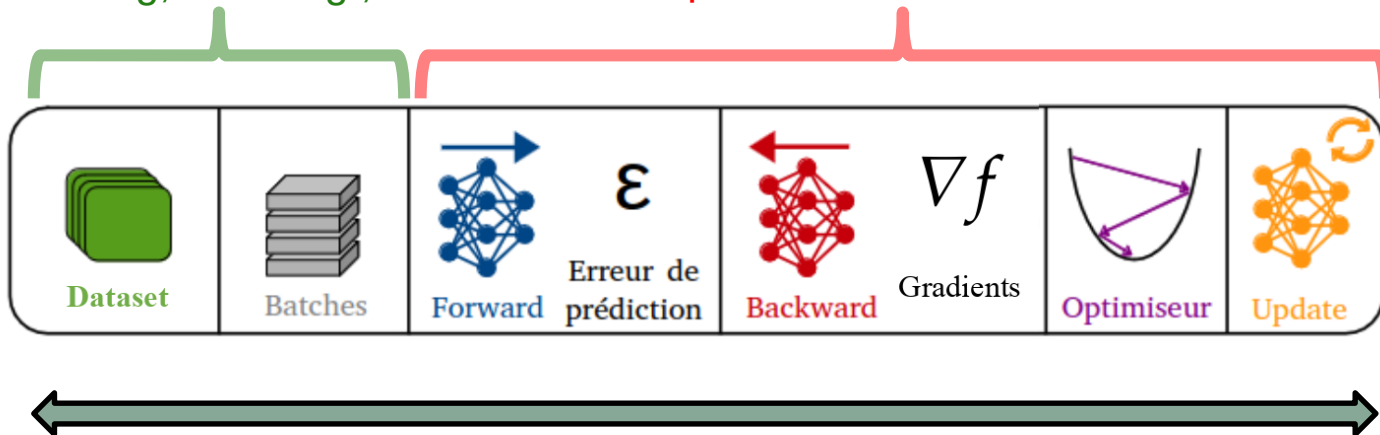
Training step

Dataloader

Preprocessing, Batching, ...

Training

Mixed precision, distribution, Metrics, ...



on
CPU

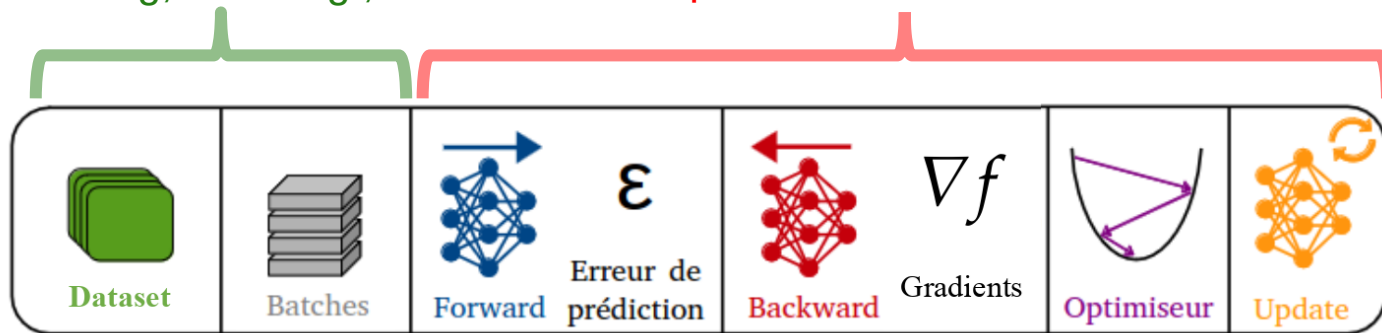
Training step – GPU Acceleration

Dataloader

Preprocessing, Batching, ...

Training

Mixed precision, distribution, Metrics, ...



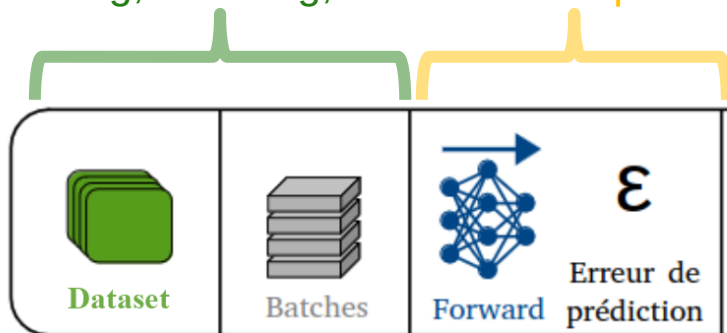
Validation step

Dataloader

Preprocessing, Batching, ...

Validation

Mixed precision, distribution, Metrics, ...



on
CPU

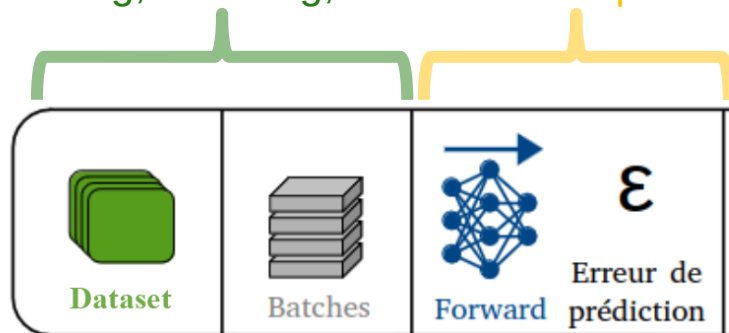
Validation step – GPU Acceleration

Dataloader

Preprocessing, Batching, ...

Validation

Mixed precision, distribution, Metrics, ...



on
CPU



to GPU

Code dlojz.py Review

Import

argparse : arguments

→ Instantiation

Model, distribution, optimizer, ...

→ Dataloader

Preprocessing, Batching, ...

Instantiation

→ Training

Mixed precision, distribution, ...

Validation

→ Mixed precision, distribution, ...

Checkpoint & report

Runner

dlojz.py – Import & run

```
import os
import contextlib
import argparse
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
from torch.utils.checkpoint import checkpoint_sequential
import torch
import numpy as np
import apex

import idr_torch
from dlojz_chrono import Chronometer
from torchmetrics.aggregation import MeanMetric
from torchmetrics.classification import MulticlassAccuracy

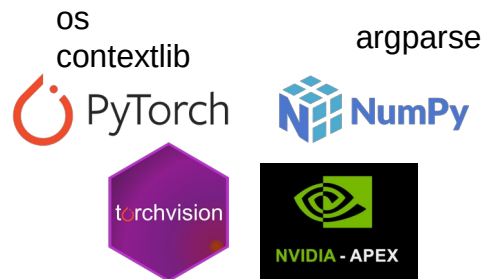
import random
random.seed(123)
np.random.seed(123)
torch.manual_seed(123)
```

reproducibility

idr_torch (JZ users)
distribution utils for Jean Zay

```
if __name__ == '__main__':
    # display info
    if idr_torch.rank == 0:
        print(">>> Training on ", len(idr_torch.hostnames), " nodes and ", idr_torch.size, " processes")
    train()
```

Import libraries



Chronometer (DLO-JZ)

time log & home profiler



TorchMetrics

Created by Lightning AI



```
28 #*****
29 def train():
30     parser = argparse.ArgumentParser()
31     parser.add_argument('-b', '--batch-s
```

dlojz.py - arguments parser

```
## import ... ## Add here the libraries to import
```

```
VAL_BATCH_SIZE=256
```

```
def train():
    parser = argparse.ArgumentParser()
    parser.add_argument('-b', '--batch-size', default=128, type=int,
                        help='batch size per GPU')
    parser.add_argument('-e', '--epochs', default=1, type=int,
                        help='number of total epochs to run')
    parser.add_argument('--image-size', default=224, type=int,
                        help='Image size')
    parser.add_argument('--lr', default=0.1, type=float,
                        help='learning rate')
    parser.add_argument('--wd', default=0., type=float,
                        help='weight decay')
    parser.add_argument('--mom', default=0.9, type=float,
                        help='momentum')
    parser.add_argument('--test', default=False, action='store_true',
                        help='Test 50 iterations')
    parser.add_argument('--test-nsteps', default='50', type=int,
                        help='the number of steps in test mode')
    parser.add_argument('--num-workers', default=10, type=int,
                        help='num workers in dataloader')
    parser.add_argument('--persistent-workers', default=True, action=argparse.BooleanOptionalAction,
                        help='activate persistent workers in dataloader')
    parser.add_argument('--pin-memory', default=True, action=argparse.BooleanOptionalAction,
                        help='activate pin memory option in dataloader')
    parser.add_argument('--non-blocking', default=True, action=argparse.BooleanOptionalAction,
                        help='activate asynchronous GPU transfer')
    parser.add_argument('--prefetch-factor', default=3, type=int,
                        help='prefetch factor in dataloader')
    parser.add_argument('--drop-last', default=False, action=argparse.BooleanOptionalAction,
                        help='activate drop last option in dataloader')
```

```
## Add parser arguments
```

```
args = parser.parse_args()
```

```
Python 3.7.14 [Anaconda, Inc.] (default, Nov 8 2017, 13:00:45) [MSC v.1500 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
>>> print("Hello dear!!!")
Hello dear!!!
>>>
```

Configurable Arguments :

- batch-size : batch size per GPU
- epochs : number of epochs
- image-size : image size

Optimizer :

- lr : learning rate
- wd : weight decay
- mom : momentum

Modes spéciaux :

- test : test mode
- test-nsteps : n steps for test mode

Optimisation du DataLoader :

- num-workers
- persistent-workers
- pin-memory
- non-blocking
- prefetch-factor
- drop-last

dlojz.py - instantiation

```
## chronometer initialisation
chrono = Chronometer()

# define model
model = models.resnet152()
archi_model = 'Resnet-152'

if idr_torch.rank == 0: print(f'model: {archi_model}')
if idr_torch.rank == 0: print('number of parameters: {}'.format(sum([p.numel()
                                                                    for p in model.parameters()])))

# distribute batch size (mini-batch)
num_replica = idr_torch.size
mini_batch_size = args.batch_size
global_batch_size = mini_batch_size * num_replica

if idr_torch.rank == 0:
    print(f'global batch size: {global_batch_size} - mini batch size: {mini_batch_size}')

# define loss function (criterion) and optimizer
criterion = torch.nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = torch.optim.SGD(model.parameters(), args.lr, momentum=args.mom, weight_decay=args.wd)

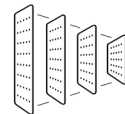
if idr_torch.rank == 0: print(f'Optimizer: {optimizer}')

# define metrics
train_metric = {}
train_metric['loss'] = MeanMetric()
train_metric['acc'] = MulticlassAccuracy(num_classes=1000, average='micro')
val_metric = {}
val_metric['loss'] = MeanMetric()
val_metric['acc'] = MulticlassAccuracy(num_classes=1000, average='micro')

#LR scheduler to accelerate the training time
scheduler = torch.optim.lr_scheduler.OneCycleLR(optimizer, max_lr=args.lr,
                                                steps_per_epoch=N_batch, epochs=args.epochs)
```



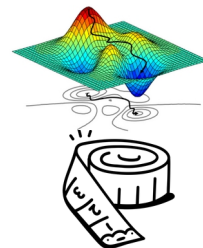
Chronometer



model : Resnet-152

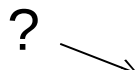
mini batch size $\longleftrightarrow$ global batch size

CrossEntropyLoss

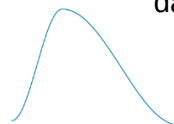


SGD Optimizer

Metric



need N_{batch} , given by
dataloader



LR scheduler

dlojz.py - DataLoader

```
##### DATALOADER #####
# Define a transform to pre-process the training images.

if idr_torch.rank == 0: print(f'DATALOADER {args.num_workers} {args.persistent_workers} {args.pin_memory}')

transform = transforms.Compose([
    transforms.RandomResizedCrop(args.image_size), # Random resize - Data Augmentation
    transforms.RandomHorizontalFlip(),             # Horizontal Flip - Data Augmentation
    transforms.ToTensor(),                         # convert the PIL Image to a tensor
    transforms.Normalize(mean=(0.485, 0.456, 0.406),
                        std=(0.229, 0.224, 0.225))
])

train_dataset = torchvision.datasets.ImageNet(root=os.environ['ALL_CCFRSCRATCH']+'/imagenet',
                                             transform=transform)

train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
                                           batch_size=mini_batch_size,
                                           shuffle=True,
                                           num_workers=args.num_workers,
                                           persistent_workers=args.persistent_workers,
                                           pin_memory=args.pin_memory,
                                           prefetch_factor=args.prefetch_factor,
                                           drop_last=args.drop_last)

val_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(224),
    transforms.ToTensor(), # convert the PIL Image to a tensor
    transforms.Normalize(mean=(0.485, 0.456, 0.406),
                        std=(0.229, 0.224, 0.225))]

val_dataset = torchvision.datasets.ImageNet(root=os.environ['ALL_CCFRSCRATCH']+'/imagenet', split='val',
                                           transform=val_transform)

val_loader = torch.utils.data.DataLoader(dataset=val_dataset,
                                         batch_size=VAL_BATCH_SIZE,
                                         shuffle=False,
                                         num_workers=args.num_workers,
                                         persistent_workers=args.persistent_workers,
                                         pin_memory=args.pin_memory,
                                         prefetch_factor=args.prefetch_factor,
                                         drop_last=args.drop_last)

N_batch = len(train_loader)
N_val_batch = len(val_loader)
N_val = len(val_dataset)
```

train dataset :

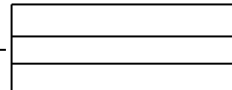
RandomResizedCrop
RandomHorizontalFlip
+ Normalize



Shuffling

DataLoader
optimization

spawn
n



batch

validation dataset :

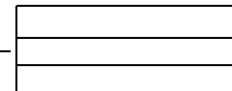
Resize
CenterCrop
+ Normalize



no shuffling

DataLoader
optimization

spawn
n



batch

dlojz.py - Training

```
chrono.start()

#### TRAINING #####
for epoch in range(args.epochs):

    if args.test: chrono.next_iter()
    if idr_torch.rank == 0: chrono.tac_time(clear=True)

    for i, (images, labels) in enumerate(train_loader):

        csteps = i + 1 + epoch * N_batch
        if args.test and csteps > args.test_nsteps: break
        if i == 0 and idr_torch.rank == 0:
            print(f'image batch shape : {images.size()}')

        if args.test: chrono.forward()

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)

        if args.test: chrono.backward()

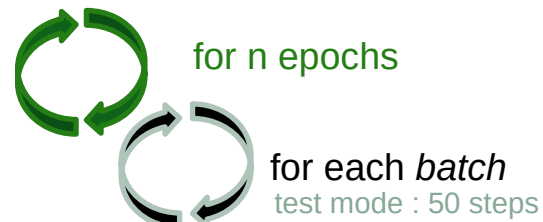
        loss.backward()
        optimizer.step()

        # Metric mesurement
        train_metric['loss'].update(loss)
        train_metric['acc'].update(outputs, labels)

        if args.test: chrono.update()

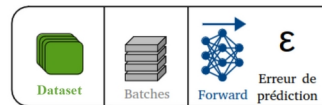
    if ((i + 1) % (N_batch//10) == 0 or i == N_batch - 1):
        train_loss, accuracy = train_metric['loss'].compute(), train_metric['acc'].compute()
        train_metric['loss'].reset(), train_metric['acc'].reset()
        if idr_torch.rank == 0:
            print('Epoch [{}]/[{}], Step [{}]/[{}], Time: {:.3f}, Loss: {:.4f}, Acc:{:.4f}'.format(
                epoch + 1, args.epochs, i+1, N_batch,
                chrono.tac_time(), train_loss, accuracy))

    # scheduler update
    scheduler.step()
```

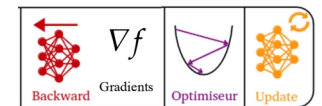


CPU compute by default !!

```
optimizer.zero_grad()
outputs = model(images)
loss = criterion(outputs, labels)
```



```
loss.backward()
optimizer.step()
```



Aggregate the metrics (loss, accuracy)
10x per epoch, compute and print the metrics

Lo 10x per epoch

g
Step up LR scheduler

dlojz.py - Validation

```
#### VALIDATION #####
if ((i == N_batch - 1) or (args.test and i==args.test_steps-1)) :

    chrono.validation()
    model.eval()

    for iv, (val_images, val_labels) in enumerate(val_loader):

        # Runs the forward pass with no grad mode.
        with torch.no_grad():
            val_outputs = model(val_images)
            val_loss = criterion(val_outputs, val_labels)

            val_metric['loss'].update(val_loss)
            val_metric['acc'].update(val_outputs, val_labels)

        if args.test and iv >= 20: break

    val_loss, val_accuracy = val_metric['loss'].compute(), val_metric['acc'].compute()
    val_metric['loss'].reset(), val_metric['acc'].reset()

    model.train()
    chrono.validation()
    if not args.test and idr_torch.rank == 0:
        print('##EVALUATION STEP##')
        print('Epoch [{}/{}], Validation Loss: {:.4f}, Validation Accuracy: {:.4f}'.format(
            epoch + 1, args.epochs, val_loss, val_accuracy))
        print(">>> Validation complete in: " + str(chrono.val_time))

#### END OF VALIDATION #####
```

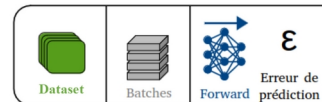


after each epoch
(or at the end of test mode)



for each *batch* of validation
(test mode : 20 steps)

```
# Runs the forward pass with no grad mode.
with torch.no_grad():
    val_outputs = model(val_images)
    loss = criterion(val_outputs, val_labels)
```



Aggregate the metrics (loss, accuracy)

when it is over:



compute & Log

dlojz.py – Checkpoint & Report

```
chrono.stop()
if idr_torch.rank == 0:
    chrono.display()
    print(">>> Number of batch per epoch: {}".format(N_batch))
    print(f'Max Memory Allocated {torch.cuda.max_memory_allocated()} Bytes')

# Save last checkpoint
if not args.test and idr_torch.rank == 0:
    checkpoint_path = f"checkpoints/{os.getenv('SLURM_JOBID')}__{global_batch_size}.pt"
    torch.save(model.state_dict(), checkpoint_path)
    print("Last epoch checkpointed to " + checkpoint_path)
```

Log + Chronometer Display

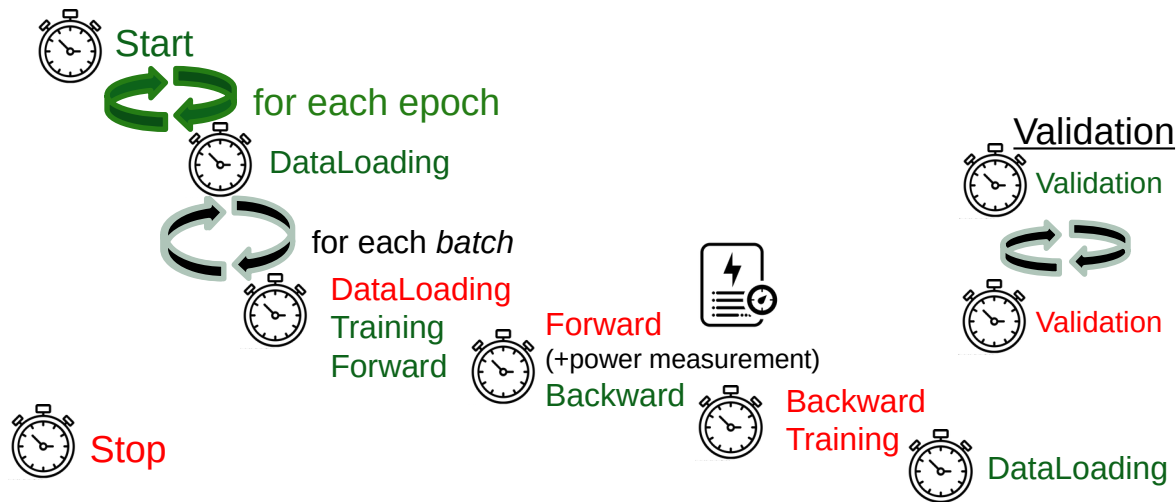
Checkpoint at the end of training (=> not in test mode)



pre-trained model

```

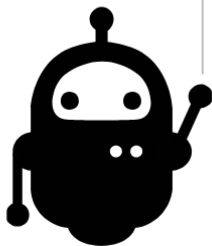
1  #!/usr/bin/perl
2  #
3  # This script is a part of the Perl module 'Text::CSV_XS'.
4  # It is used to convert a CSV file to a JSON file.
5  #
6  # Usage: perl csv2json.pl input.csv output.json
7  #
8  # Options:
9  # -i, --input FILENAME Input CSV file
10 # -o, --output FILENAME Output JSON file
11 # -t, --type TYPE Output JSON type (array or object)
12 # -s, --separator SEPARATOR CSV separator (comma or semicolon)
13 # -e, --escape ESCAPE CSV escape character (double quote or backslash)
14 # -h, --help Display this help message
15 #
16 # Example:
17 # perl csv2json.pl data.csv data.json
18 #
19 # Copyright (c) 2000-2010, Mark R. Miller
20 # All rights reserved.
21 #
22 # This program is free software; you can redistribute it and/or
23 # modify it under the terms of the GNU General Public License
24 # as published by the Free Software Foundation; either version 2
25 # of the License, or (at your option) any later version.
26 #
27 # This program is distributed in the hope that it will be useful,
28 # but WITHOUT ANY WARRANTY; without even the implied warranty of
29 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
30 # GNU General Public License for more details.
31 #
32 # You should have received a copy of the GNU General Public License
33 # along with this program; if not, write to the Free Software
34 # Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301,
35 # USA.
36 #
37 #
38 #
39 #
40 #
41 #
42 #
43 #
44 #
45 #
46 #
47 #
48 #
49 #
50 #
51 #
52 #
53 #
54 #
55 #
56 #
57 #
58 #
59 #
60 #
61 #
62 #
63 #
64 #
65 #
66 #
67 #
68 #
69 #
70 #
71 #
72 #
73 #
74 #
75 #
76 #
77 #
78 #
79 #
80 #
81 #
82 #
83 #
84 #
85 #
86 #
87 #
88 #
89 #
90 #
91 #
92 #
93 #
94 #
95 #
96 #
97 #
98 #
99 #
100 #
101 #
102 #
103 #
104 #
105 #
106 #
107 #
108 #
109 #
110 #
111 #
112 #
113 #
114 #
115 #
116 #
117 #
118 #
119 #
120 #
121 #
122 #
123 #
124 #
125 #
126 #
127 #
128 #
129 #
130 #
131 #
132 #
133 #
134 #
135 #
136 #
137 #
138 #
139 #
140 #
141 #
142 #
143 #
144 #
145 #
146 #
147 #
148 #
149 #
150 #
151 #
152 #
153 #
154 #
155 #
156 #
157 #
158 #
159 #
160 #
161 #
162 #
163 #
164 #
165 #
166 #
167 #
168 #
169 #
170 #
171 #
172 #
173 #
174 #
175 #
176 #
177 #
178 #
179 #
180 #
181 #
182 #
183 #
184 #
185 #
186 #
187 #
188 #
189 #
190 #
191 #
192 #
193 #
194 #
195 #
196 #
197 #
198 #
199 #
200 #
201 #
202 #
203 #
204 #
205 #
206 #
207 #
208 #
209 #
210 #
211 #
212 #
213 #
214 #
215 #
216 #
217 #
218 #
219 #
220 #
221 #
222 #
223 #
224 #
225 #
226 #
227 #
228 #
229 #
230 #
231 #
232 #
233 #
234 #
235 #
236 #
237 #
238 #
239 #
240 #
241 #
242 #
243 #
244 #
245 #
246 #
247 #
248 #
249 #
250 #
251 #
252 #
253 #
254 #
255 #
256 #
257 #
258 #
259 #
260 #
261 #
262 #
263 #
264 #
265 #
266 #
267 #
268 #
269 #
270 #
271 #
272 #
273 #
274 #
275 #
276 #
277 #
278 #
279 #
280 #
281 #
282 #
283 #
284 #
285 #
286 #
287 #
288 #
289 #
290 #
291 #
292 #
293 #
294 #
295 #
296 #
297 #
298 #
299 #
300 #
301 #
302 #
303 #
304 #
305 #
306 #
307 #
308 #
309 #
310 #
311 #
312 #
313 #
314 #
315 #
316 #
317 #
318 #
319 #
320 #
321 #
322 #
323 #
324 #
325 #
326 #
327 #
328 #
329 #
330 #
331 #
332 #
333 #
334 #
335 #
336 #
337 #
338 #
339 #
340 #
341 #
342 #
343 #
344 #
345 #
346 #
347 #
348 #
349 #
350 #
351 #
352 #
353 #
354 #
355 #
356 #
357 #
358 #
359 #
360 #
361 #
362 #
363 #
364 #
365 #
366 #
367 #
368 #
369 #
370 #
371 #
372 #
373 #
374 #
375 #
376 #
377 #
378 #
379 #
380 #
381 #
382 #
383 #
384 #
385 #
386 #
387 #
388 #
389 #
390 #
391 #
392 #
393 #
394 #
395 #
396 #
397 #
398 #
399 #
400 #
401 #
402 #
403 #
404 #
405 #
406 #
407 #
408 #
409 #
410 #
411 #
412 #
413 #
414 #
415 #
416 #
417 #
418 #
419 #
420 #
421 #
422 #
423 #
424 #
425 #
426 #
427 #
428 #
429 #
430 #
431 #
432 #
433 #
434 #
435 #
436 #
437 #
438 #
439 #
440 #
441 #
442 #
443 #
444 #
445 #
446 #
447 #
448 #
449 #
450 #
451 #
452 #
453 #
454 #
455 #
456 #
457 #
458 #
459 #
460 #
461 #
462 #
463 #
464 #
465 #
466 #
467 #
468 #
469 #
470 #
471 #
472 #
473 #
474 #
475 #
476 #
477 #
478 #
479 #
480 #
481 #
482 #
483 #
484 #
485 #
486 #
487 #
488 #
489 #
490 #
491 #
492 #
493 #
494 #
495 #
496 #
497 #
498 #
499 #
500 #
501 #
502 #
503 #
504 #
505 #
506 #
507 #
508 #
509 #
510 #
511 #
512 #
513 #
514 #
515 #
516 #
517 #
518 #
519 #
520 #
521 #
522 #
523 #
524 #
525 #
526 #
527 #
528 #
529 #
530 #
531 #
532 #
533 #
534 #
535 #
536 #
537 #
538 #
539 #
540 #
541 #
542 #
543 #
544 #
545 #
546 #
547 #
548 #
549 #
550 #
551 #
552 #
553 #
554 #
555 #
556 #
557 #
558 #
559 #
560 #
561 #
562 #
563 #
564 #
565 #
566 #
567 #
568 #
569 #
570 #
571 #
572 #
573 #
574 #
575 #
576 #
577 #
578 #
579 #
580 #
581 #
582 #
583 #
584 #
585 #
586 #
587 #
588 #
589 #
590 #
591 #
592 #
593 #
594 #
595 #
596 #
597 #
598 #
599 #
600 #
601 #
602 #
603 #
604 #
605 #
606 #
607 #
608 #
609 #
610 #
611 #
612 #
613 #
614 #
615 #
616 #
617 #
618 #
619 #
620 #
621 #
622 #
623 #
624 #
625 #
626 #
627 #
628 #
629 #
630 #
631 #
632 #
633 #
634 #
635 #
636 #
637 #
638 #
639 #
640 #
641 #
642 #
643 #
644 #
645 #
646 #
647 #
648 #
649 #
650 #
651 #
652 #
653 #
654 #
655 #
656 #
657 #
658 #
659 #
660 #
661 #
662 #
663 #
664 #
665 #
666 #
667 #
668 #
669 #
670 #
671 #
672 #
673 #
674 #
675 #
676 #
677 #
678 #
679 #
680 #
681 #
682 #
683 #
684 #
685 #
686 #
687 #
688 #
689 #
690 #
691 #
692 #
693 #
694 #
695 #
696 #
697 #
698 #
699 #
700 #
701 #
702 #
703 #
704 #
705 #
706 #
707 #
708 #
709 #
710 #
711 #
712 #
713 #
714 #
715 #
716 #
717 #
718 #
719 #
720 #
721 #
722 #
723 #
724 #
725 #
726 #
727 #
728 #
729 #
730 #
731 #
732 #
733 #
734 #
735 #
736 #
737 #
738 #
739 #
740 #
741 #
742 #
743 #
744 #
745 #
746 #
747 #
748 #
749 #
750 #
751 #
752 #
753 #
754 #
755 #
756 #
757 #
758 #
759 #
760 #
761 #
762 #
763 #
764 #
765 #
766 #
767 #
768 #
769 #
770 #
771 #
772 #
773 #
774 #
775 #
776 #
777 #
778 #
7
```



Display

```
def display(self, val_steps):
    if self.rank == 0:
        print(">>> Training complete in: " + str(datetime.now() - self.start_proc))
        if self.test:
            print(">>> Training performance time: min {} avg {} seconds (+/- {})".format(np.min(self.time_perf_train[1:]), np.median(self.time_perf_train[1:]), np.std(self.time_perf_train[1:])))
            print(">>> Loading performance time: min {} avg {} seconds (+/- {})".format(np.min(self.time_perf_load[1:]), np.mean(self.time_perf_load[1:]), np.std(self.time_perf_load[1:])))
            print(">>> Forward performance time: {} seconds (+/- {})".format(np.mean(self.time_perf_forward[1:]), np.std(self.time_perf_forward[1:])))
            print(">>> Backward performance time: {} seconds (+/- {})".format(np.mean(self.time_perf_backward[1:]), np.std(self.time_perf_backward[1:])))
            if len(self.power)>0: print(">>> Peak Power during training: {} W)".format(np.max(self.power)))
            print(">>> Validation time estimation: {}".format(self.val_time/20 * val_steps))
            print(">>> Sortie trace #####")
            print(">>>JSON", json.dumps({'GPU process - Forward/Backward':self.time_perf_train, 'CPU process - Dataloader':self.time_perf_load}))
```

TP0 : Préparation de l'environnement



- Lancer un terminal et faire les copies nécessaires

```
local:~$ ssh jean-zay  
  
jz:~$ cd $WORK  
jz:~$ git clone https://github.com/IDRIS-CNRS/DLO-JZ.git
```

- Lancer firefox
- Accéder à jupyterhub.idris.fr

TP0 : Accès et prise en main de JupyterHub

- Se connecter avec vos identifiants de formation

- Lancer une instance

List of JupyterLab instances

Every user may have 10 JupyterLab server(s) with names. This allows the u

DLO_TP	Add New JupyterLab Instance	
Instance name	URL	Node type

- Sélectionner le spawner 'Interactive'

Interactive	SLURM
-------------	-------

- Remplir la configuration

- Start

JupyterLab instance will be launched on a Jean Zay frontal node. Globally, the resources are limited to one CPU and 5 GB of memory for each user.

Time (--time) (in hours)

8

Notebook directory (--ServerApp.notebook_dir)

/gpfswork/idris/for/cfor032

Root directory of the JupyterLab file explorer is also set to this path

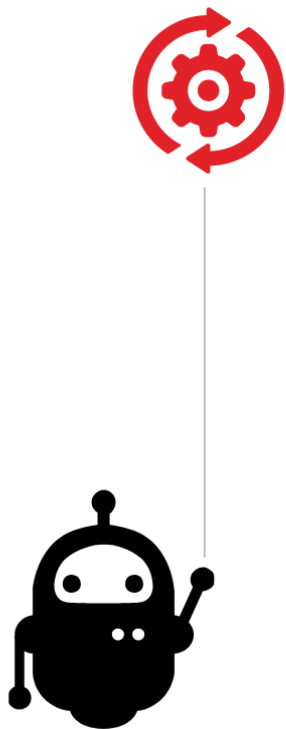
Environment variables (one per line)

WHOAMI=JUPYTERHUB

Custom environment variables can be defined here. Subshells are not supported

Start

TP0 : Accès et prise en main du notebook



- Ouvrir le notebook DLO-JZ_Jour1.ipynb
- Choisir le kernel pytorch-gpu/py3/2.1.1 (en haut à droite) s'il n'est pas détecté automatiquement
- Choisir un pseudonyme
- Lancer un job
- Prendre en main le script de référence et les différentes fonctionnalités

GPU computing

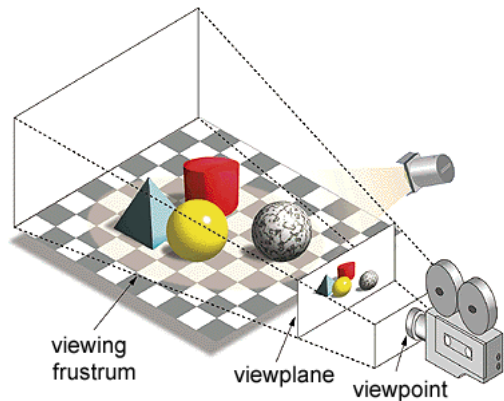
V100, A100, H100 ◀

CUDA ◀

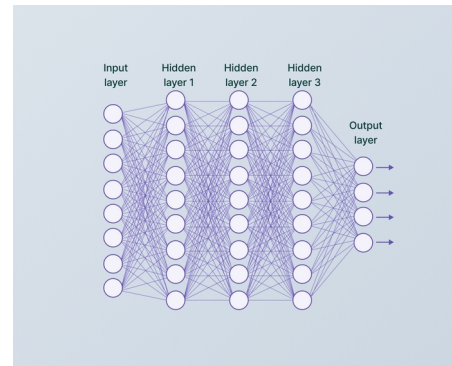
CuDNN ◀

AMP ◀

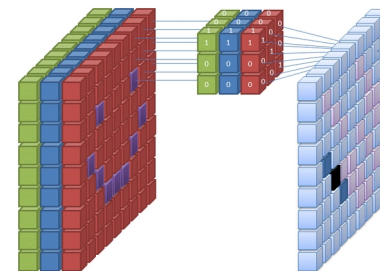
GPU computing



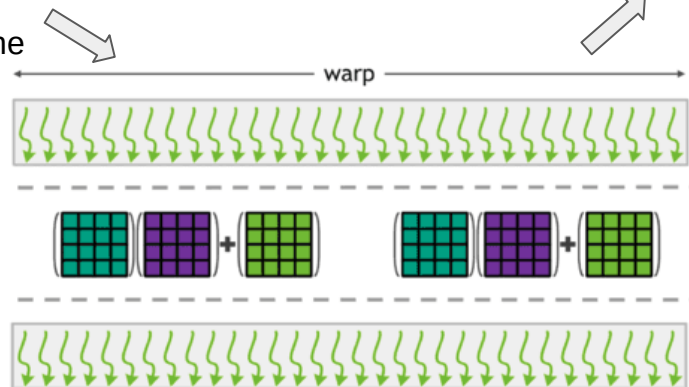
GPU Rendering & Game Graphics Pipeline



NN



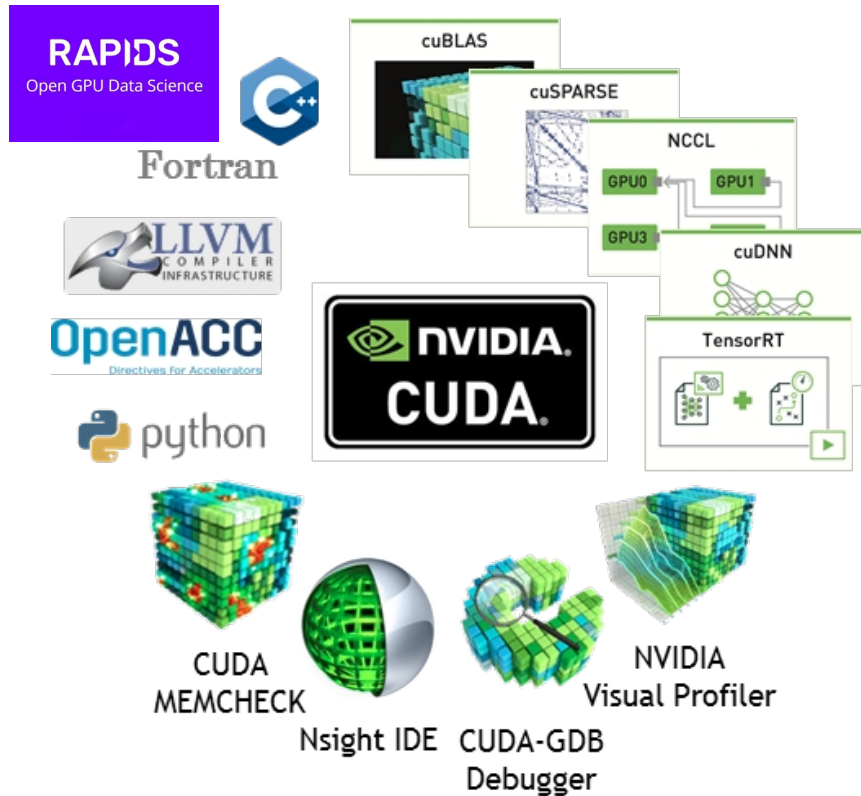
CNN



Matrix Multiply-accumulate operations



NVIDIA Galaxy



TRAINING AND INFERENCE

INFERENCE AT THE EDGE

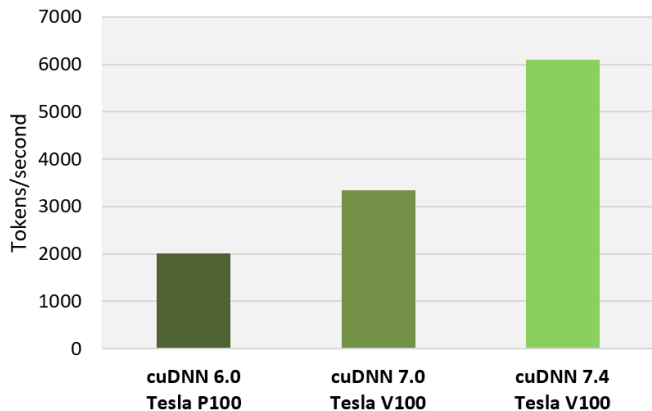
DESKTOP	DATACENTER AND CLOUD
DGX Station Titan V	DGX-2 DGX-1 Tesla V100
AUTONOMOUS MACHINES	AI SELF-DRIVING PLATFORM
Jetson TX2 Jetson TX1	DRIVE Pegasus
NVIDIA DEEP LEARNING SDK and CUDA	

Source : [NVidia](https://www.nvidia.com)

CuDNN

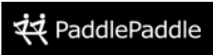


Up to 3x Faster RNN Training



TensorFlow performance (tokens/sec), Tesla P100 + cuDNN 6 (FP32) on 17.12 NGC container, Tesla V100 + cuDNN 7.0 (Mixed) on 18.02 NGC container, Tesla V100 + cuDNN 7.4 (Mixed) on 18.10 NGC container, OpenSeq2Seq (GNMT), Batch Size: 64





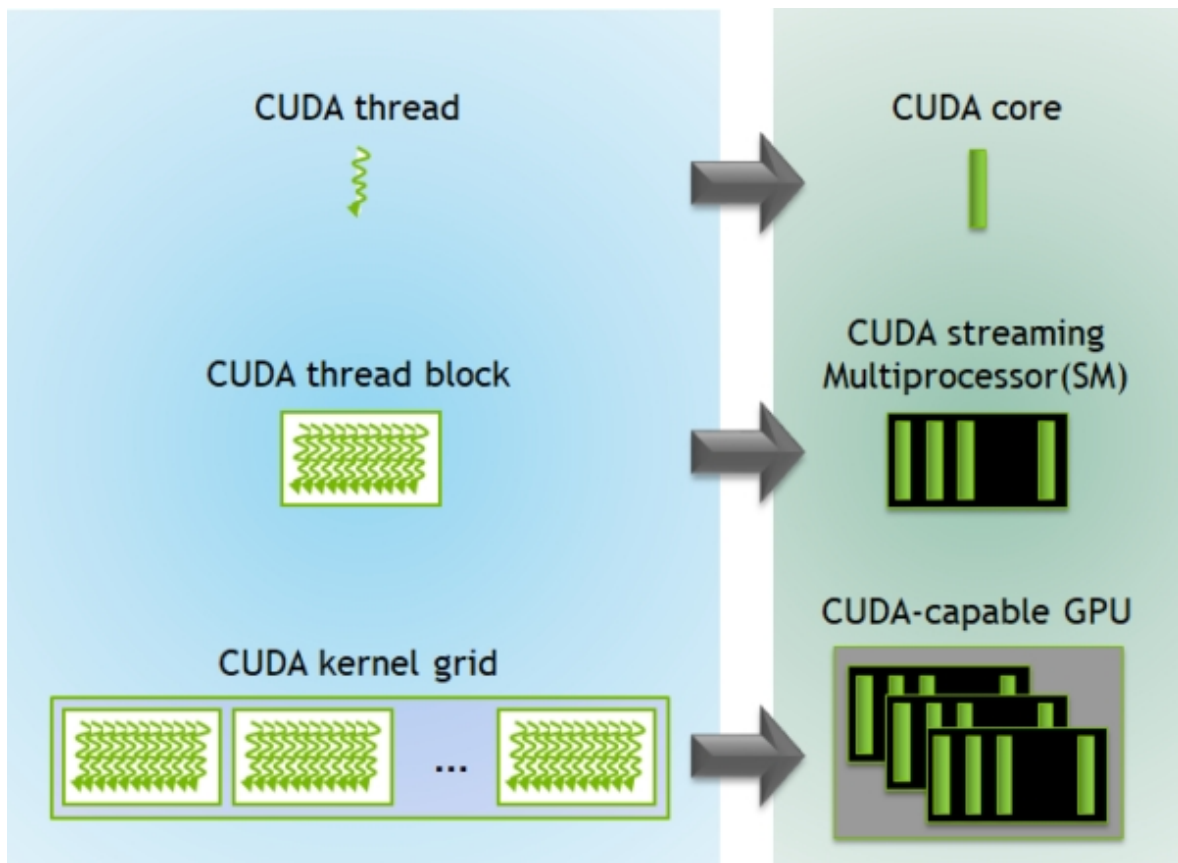
NVIDIA DEEP LEARNING SDK and CUDA

CUDA engineering for deep learning on GPU is handled by cuDNN.

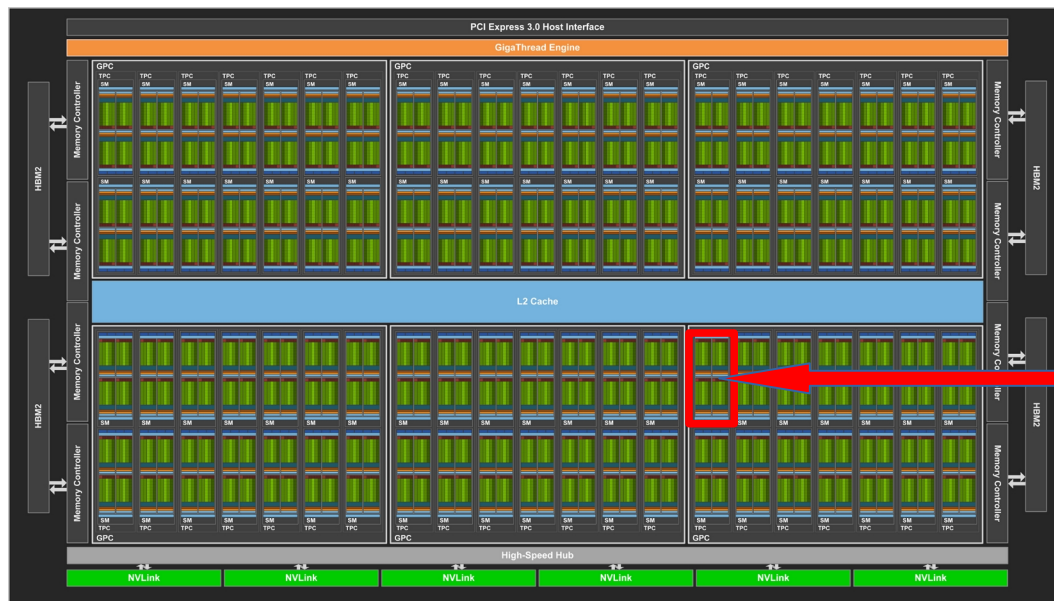
Thanks cuDNN!!

Recommendation: to optimize the use of Tensor Cores and Cuda Cores: Use tensors with dimensions (batch size, sample size, channel, layer dimension, etc.) **multiples of 8!!**

GPU computing : CUDA



V100 Architecture



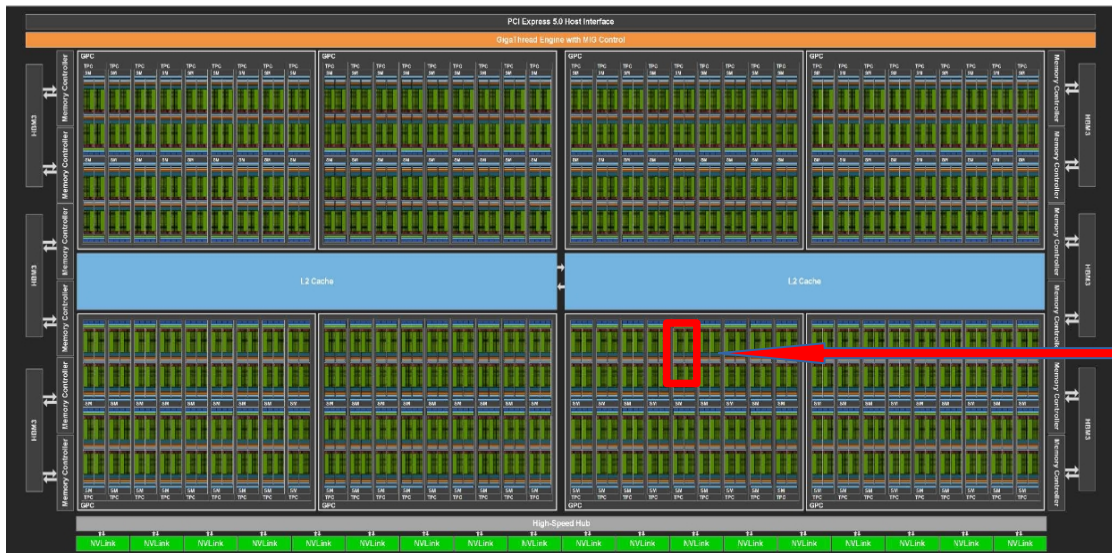
- 6 GPC
- **84** Streaming Multiprocessors (SMs)
- 5376 CUDA Cores
- **672 2eGen** Tensor Cores per full GPU

Source : [NVidia](#)



- Source : [NVidia](#)

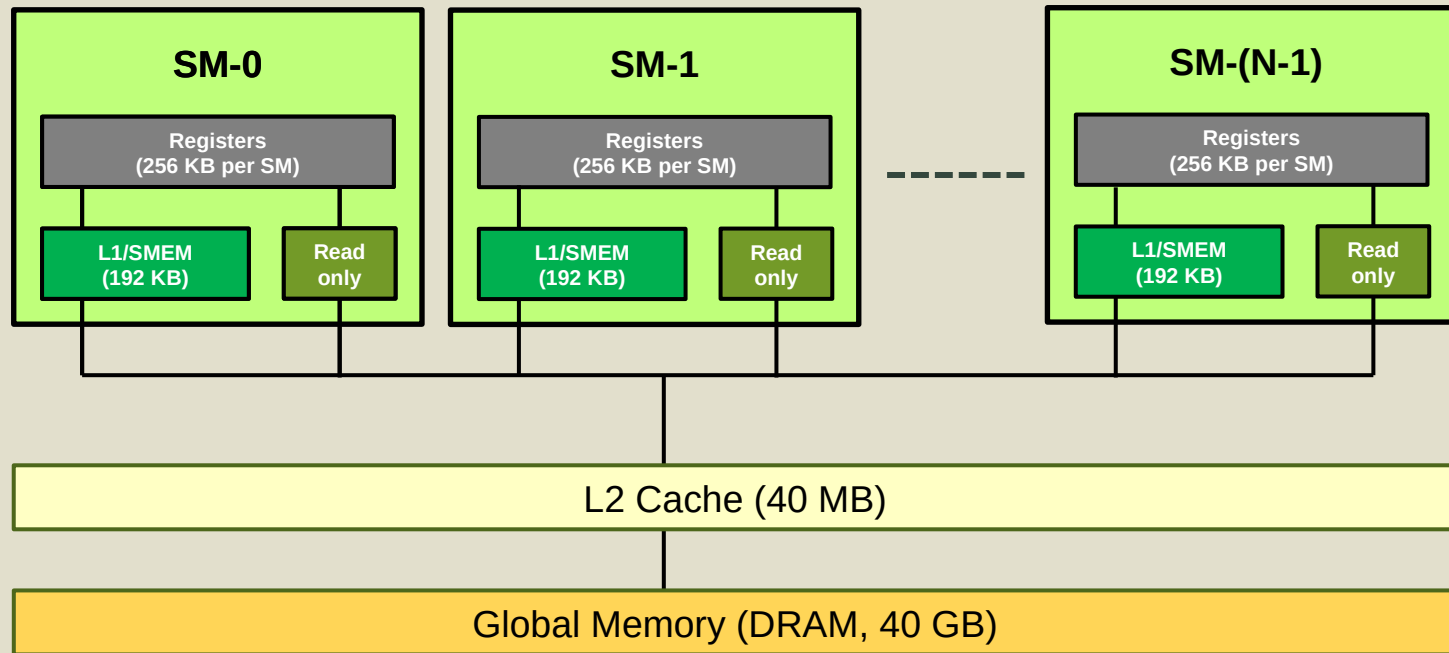
H100 Architecture

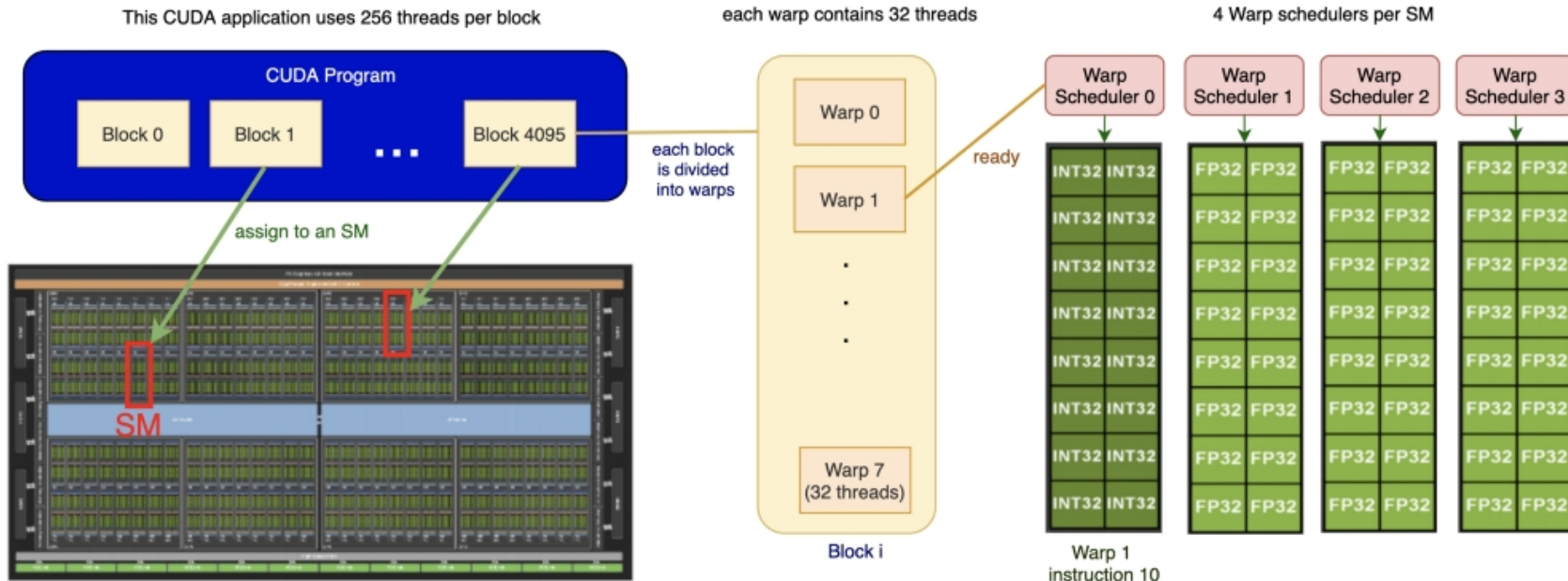


- 8 GPC
- 132 Streaming Multiprocessors (SMs)
- 16896 CUDA Cores
- 528 4eGen Tensor Cores per full GPU

Source : [NVidia](#)

Optimized memory management





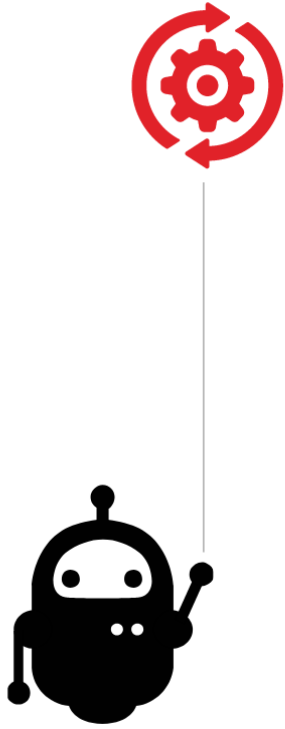
Optimization :

- Block occupancy
- Streaming dispersion

Advanced Optimization :

- Kernel Fusion to override initialization times

TP1 : Accélération GPU



- Envoyer le calcul sur le GPU
- Test Mémoire

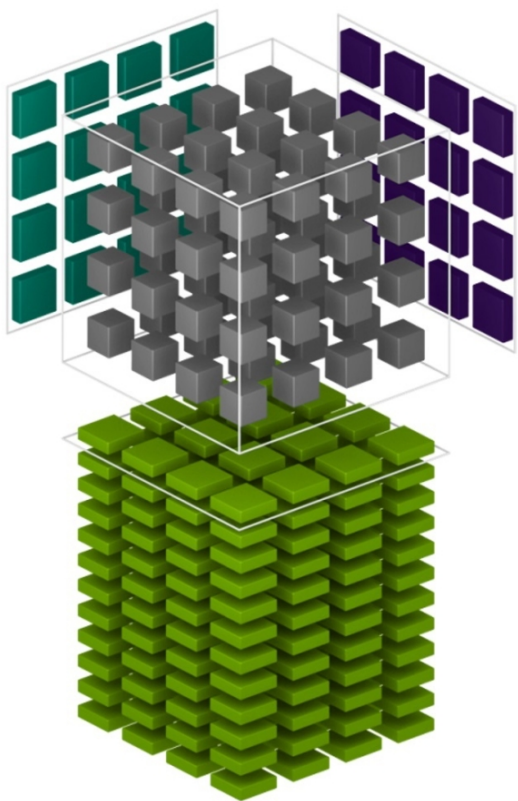
Tensor Cores

Tensor Cores ◀

Precisions ◀

AMP ◀

Channel last memory format ◀



CUDA Core are specialized for **vector computing**.

Tensor Cores are specialized for **matrix calculation**.

$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

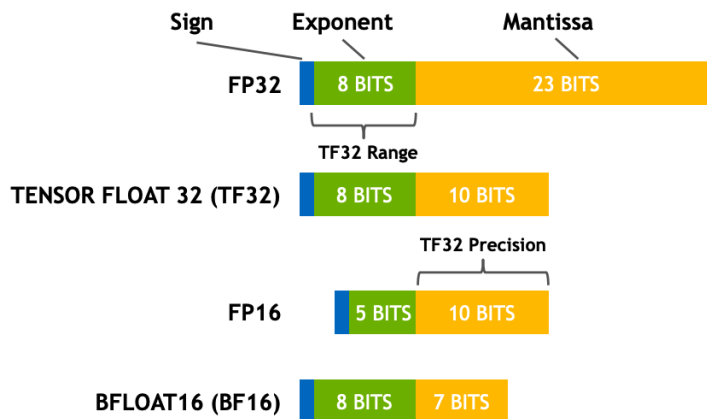
FP16 or FP32 FP16 FP16 FP16 or FP32

Each Tensor Core is capable of processing 64 operations in 1 clock time.

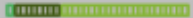
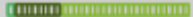
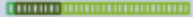
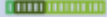
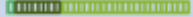
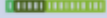
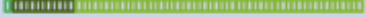
Precisions & Tensor Cores

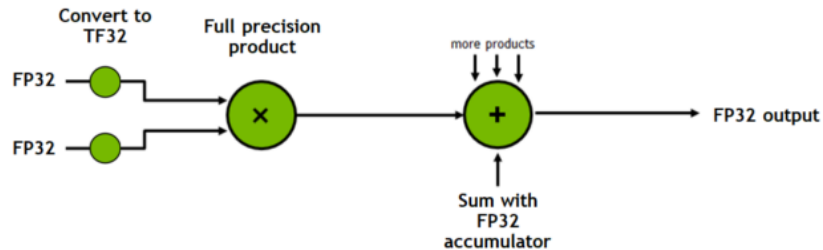
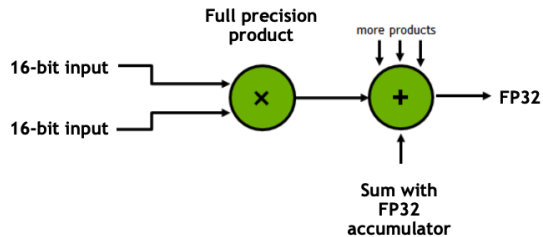


	NVIDIA H100	NVIDIA A100	NVIDIA Volta
Supported Tensor Core Precisions	FP8 , FP64, TF32, bfloat16, FP16, ...	FP64, TF32, bfloat16, FP16, INT8, INT4, INT1	FP16
Supported CUDA [®] Core Precisions	FP64, FP32, FP16, bfloat16, INT8	FP64, FP32, FP16, bfloat16, INT8	FP64, FP32, FP16, INT8

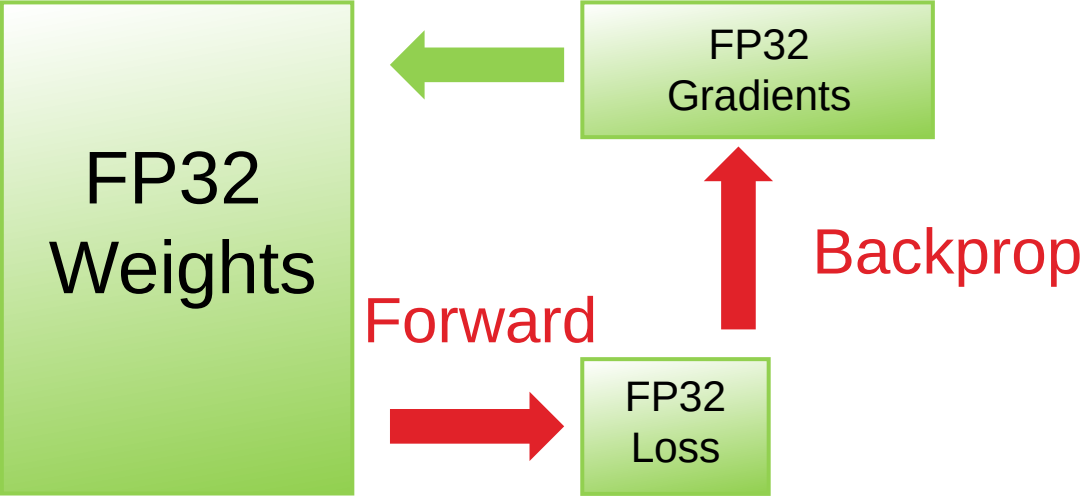


Precisions & Tensor Cores

	INPUT OPERANDS	ACCUMULATOR	TOPS	X-factor vs. FFMA	SPARSE TOPS	SPARSE X-factor vs. FFMA
V100	FP32 	FP32 	15.7	1x	-	-
	FP16 	FP32 	125	8x	-	-
A100	FP32 	FP32 	19.5	1x	-	-
	TF32 	FP32 	156	8x	312	16x
	FP16 	FP32 	312	16x	624	32x
	BF16 	FP32 	312	16x	624	32x
	FP16 	FP16 	312	16x	624	32x
	INT8 	INT32 	624	32x	1248	64x
	INT4 	INT32 	1248	64x	2496	128x
	BINARY 	INT32 	4992	256x	-	-
	IEEE FP64 		19.5	1x	-	-

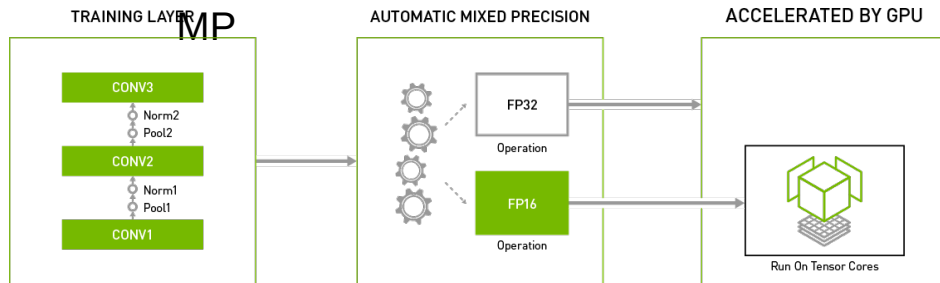


Normal Precision Training Loop

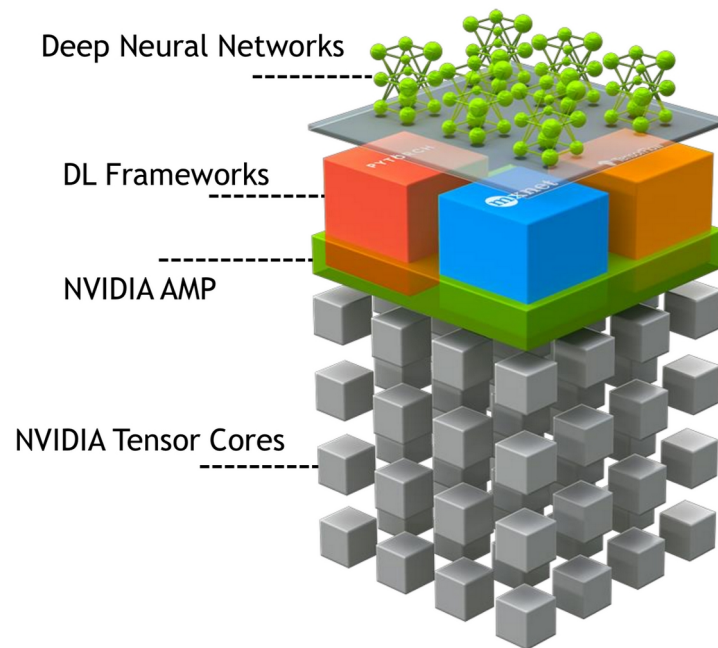


Automatic Mixed Precision

- Automatic Mixed Precision :
 - Necessary with V100 to use Tensor Core
 - The A100s use Tensor Cores with or without



- Pros:
 - **Insignificant** loss of precision for model training (gradient, loss, accuracy)
 - **Reduces memory** footprint
 - **Speeds up** calculations
- 2 steps to code :
 - Transforms eligible layers into FP16
 - Uses scaling to calculate gradients

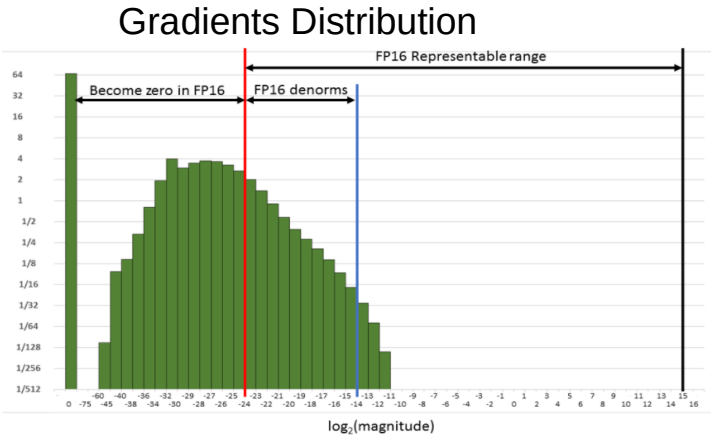


AMP FP16 with Scaler

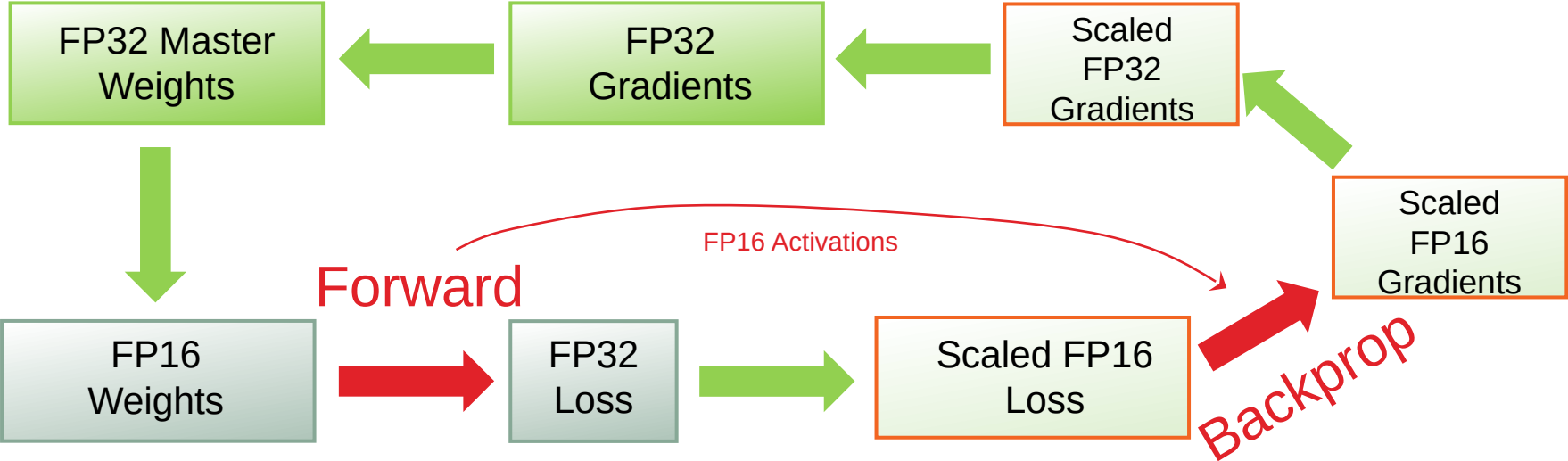
FP16



In FP16, values lower than 2^{-24} ($5.96e^{-8}$) are considered 0.



Optimizer

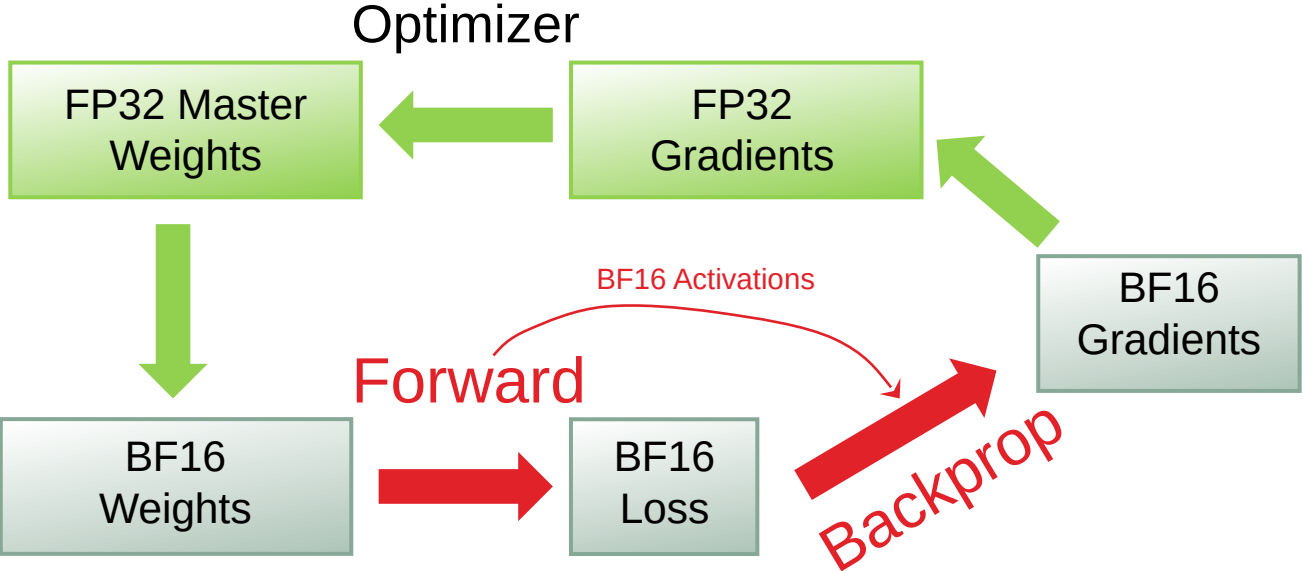


AMP BF16 without Scaler

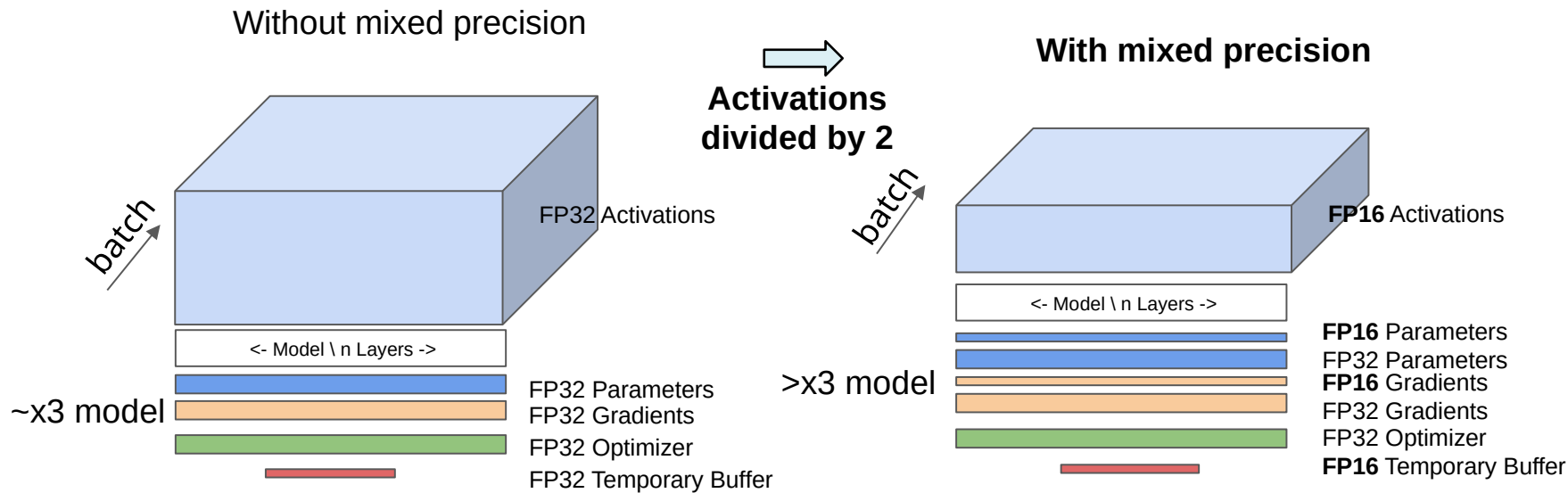
BF16



No Scaler !



Memory Footprint with Mixed Precision

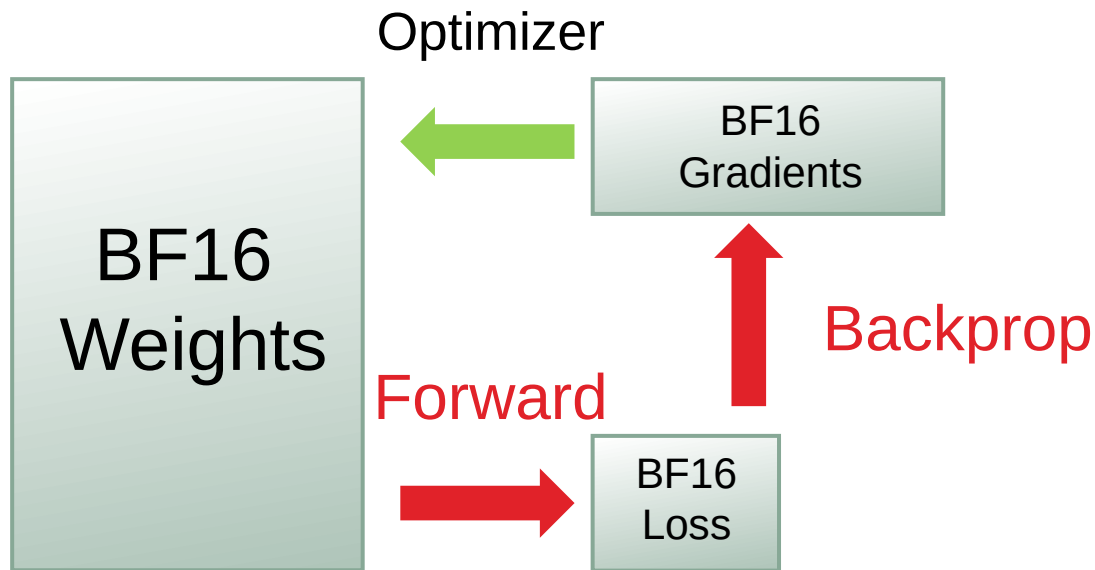


Full Degraded Precision (BF16)

BF16

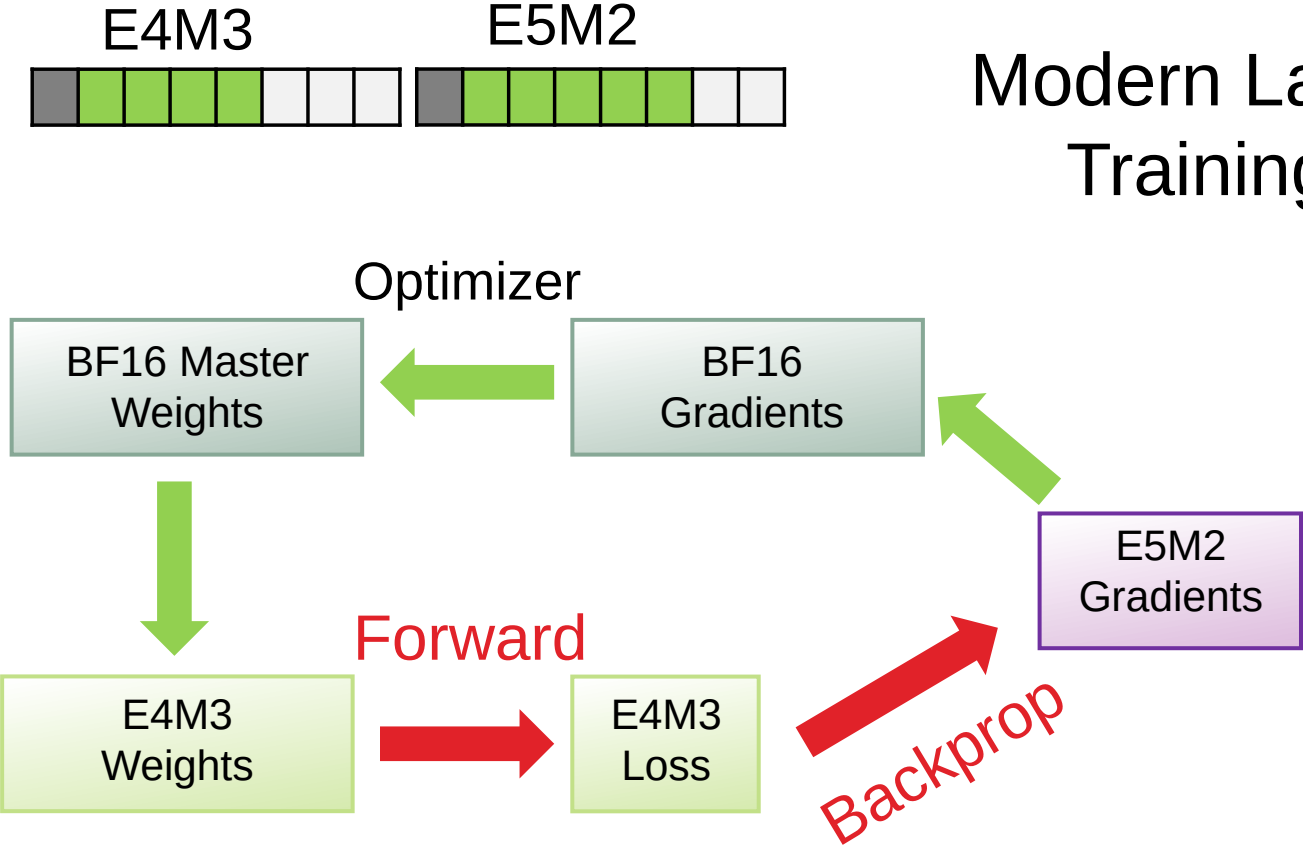


Modern Large Model
Training Way !



FP8 Mixed Precision

Modern Large Model
Training Way !



Channel last memory format

batch channel height width

NCHW

.shape()



memory contiguity by default

classic (contiguous) memory storage of NCHW tensor :

.stride()

b 0x: 0 1 2 3 4 5 6 7 8 9 a b c d e f 0 1 2 3 4 5 6 7 8 9 a b c d e f 0 1 2 3 4 5 6 7 8 9 a b c d e f

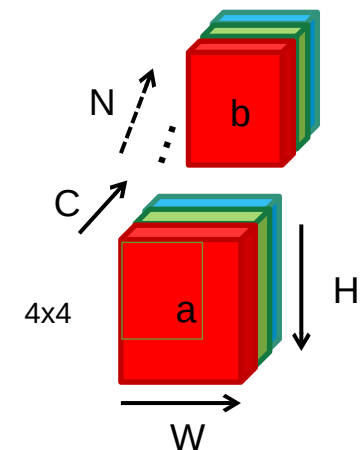
a 0x: 0 1 2 3 4 5 6 7 8 9 a b c d e f 0 1 2 3 4 5 6 7 8 9 a b c d e f 0 1 2 3 4 5 6 7 8 9 a b c d e f

Channels last memory format orders data differently:

.stride()

b 0x: 0 0 0 1 1 1 2 2 2 3 3 3 4 4 4 5 5 5 6 6 6 7 7 7 8 8 8 9 9 9 a a a b b b c c c d d d e e e f f f

a 0x: 0 0 0 1 1 1 2 2 2 3 3 3 4 4 4 5 5 5 6 6 6 7 7 7 8 8 8 9 9 9 a a a b b b c c c d d d e e e f f f



3x3

Convolution filter

TP2&3 : Automatic Mixed Precision



- Activer l'Automatic Mixed Precision
- Test Mémoire
- Activer le channel last memory format

